CHAPTER

1

The Cloud System as a Whole

Course Outcome CO1

CHAPTER OUTLINE

1.1 A Working Multi-Tier Cloud Application
1.2 Canonical Cloud-Native Architecture
1.3 Cloud Service Models
1.4 The Shared Responsibility Model
1.5 AWS, Azure and GCP Compared
1.6 End-to-End Request Tracing
1.7 AWS Global Infrastructure
1.8 Course Scope and Boundaries
Chapter Summary, Key Terms, Self-Test and Review Questions, Exercises, Hands-On Lab 1, Further Reading

LEARNING OBJECTIVES

After studying this chapter, you should be able to:

- Describe the BookStash application and trace a request from the browser to the database and back.
- Identify each tier of the canonical cloud-native architecture and the AWS service that fills it.
- Distinguish IaaS, PaaS, SaaS, FaaS and CaaS by the layers each one leaves to the customer.
- Apply the shared responsibility model to the security and reliability of a real deployment.
- Map AWS services to their closest Microsoft Azure and Google Cloud equivalents.
- Read an X-Ray trace and locate the evidence that each hop of a request leaves behind.
- Explain Regions, Availability Zones and edge locations as a hierarchy of failure domains.
- State what this course covers and where CI/CD and large-scale distributed design are treated.

In this chapter we look at a cloud system as a single working whole before we study any of its parts. We deploy and observe BookStash, a realistic multi-tier application on AWS, follow one request through every tier, and use what we see to introduce the vocabulary of the course: the canonical cloud-native architecture, service models, shared responsibility, the equivalents on other clouds, request tracing and the physical infrastructure underneath. Later chapters take each tier apart in detail.

1.1 A Working Multi-Tier Cloud Application

Most cloud textbooks begin with definitions and work outwards towards a running system. We take the opposite route. In this chapter we start with a complete application that is already deployed on Amazon Web Services (AWS), we watch a request travel through it, and only then do we name and classify the parts. The reasoning is practical. A student who has seen a page load through a CDN, a load balancer, a container and a database has a mental hook on which every later definition can hang. A student who starts from definitions has nothing to attach them to.

The application we use throughout the course is called **BookStash**. It is a small but realistic online catalogue and ordering service for a university bookstore. Readers can browse books, search by title or author, view cover images and place orders for collection at the campus store. Staff can update stock levels. The application is deliberately ordinary. It has the same moving parts as an e-commerce site, a ticketing portal or an internal HR system, and that ordinariness is what makes it useful for teaching.

1.1.1 What BookStash Does

From the reader's point of view BookStash is a single web address, https://bookstash.com, that returns pages and data quickly. Behind that address the application exposes a small HTTP interface. Table 1.1 lists the operations we refer to in this chapter.

Table 1.1 BookStash HTTP interface used in this chapter

Operation	Method	Behaviour
/api/books/{id}	GET	Returns one book as JSON. Read-heavy and cacheable.
/api/search?q=	GET	Full-text search over titles and authors. Hits the database directly.
/covers/{id}.jpg	GET	Returns a cover image. Static content, cached at the edge for a day.
/api/orders	POST	Accepts an order, places it on a queue, returns 202 Accepted.
/api/stock/{id}	PUT	Staff-only stock update. Invalidates the cached book record.
/healthz	GET	Health check used by the load balancer. Returns 200 when the task can serve.

The interface is small, but it already contains three kinds of traffic that behave very differently in a cloud system. Cover images are static and can be served from a cache anywhere in the world.

Book lookups are dynamic but repetitive, so a fast in-memory cache in front of the database pays off. Orders are writes that must not be lost, and the work that follows an order (stock reservation, confirmation email, staff notification) does not need to happen while the reader waits. We will see that each kind of traffic is handled by a different part of the architecture.

1.1.2 The Deployed System at a Glance

Figure 1.1 shows BookStash as it runs in the AWS Region ap-south-1 (Mumbai). We read the diagram from left to right, which is also the direction of a request.

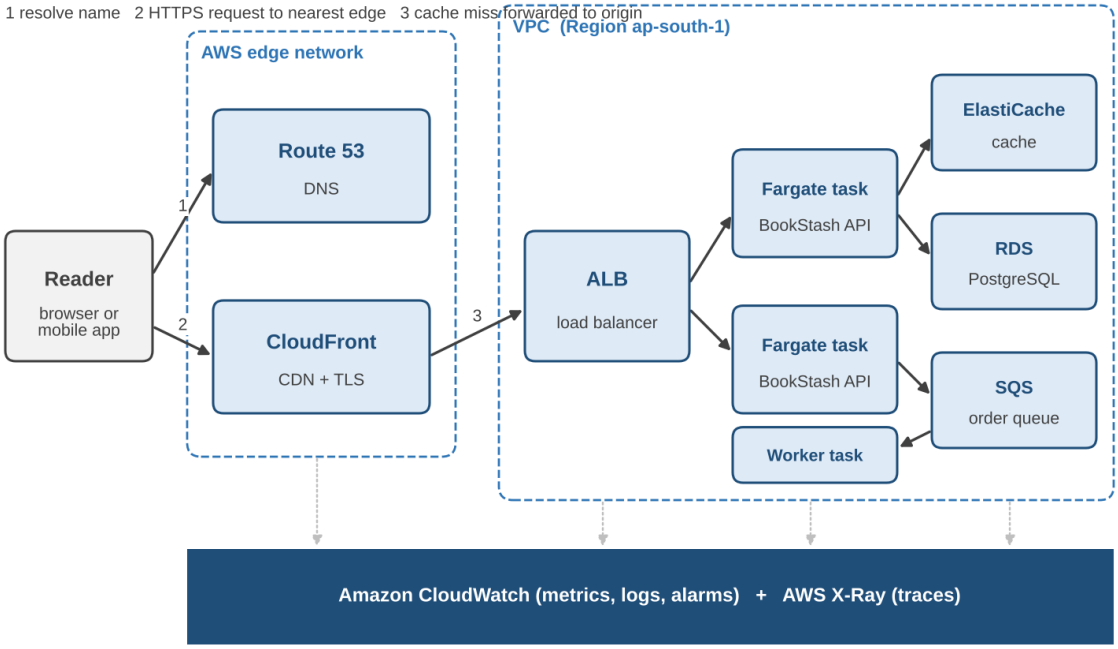


Figure 1.1 The BookStash application deployed on AWS. Edge services sit outside the VPC; the application and data tiers run inside it. CloudWatch and X-Ray observe every tier.

The reader's browser first asks **Amazon Route 53** for the address of bookstash.com. Route 53 answers with the address of the nearest **Amazon CloudFront** edge location. The browser opens an encrypted HTTPS connection to that edge location, which may be a few kilometres away rather than in Mumbai. If CloudFront already holds a fresh copy of what was asked for, it replies immediately. If not, it forwards the request over the AWS private network to the origin, which is an **Application Load Balancer (ALB)** inside a Virtual Private Cloud (VPC). The ALB picks one of several identical containers running the BookStash API on **Amazon ECS with AWS Fargate**. The container checks **Amazon ElastiCache** for the requested record, falls back to **Amazon RDS** for PostgreSQL if the cache does not have it, and returns JSON. Orders take a different path: the container writes them to an **Amazon SQS** queue and a separate worker task processes them

later. Throughout, **Amazon CloudWatch** collects metrics and logs and **AWS X-Ray** records traces that show how long each step took.

Every one of these services will receive its own chapter later in the course. For now we only need a working sense of what each box contributes, and Table 1.2 gives that in one place.

Table 1.2 What each component contributes to BookStash

Component	AWS service	Contribution to BookStash
Name resolution	Amazon Route 53	Maps bookstash.com to CloudFront using an alias record; checks origin health.
Edge delivery	Amazon CloudFront	Terminates TLS near the reader, caches covers and some API responses.
Traffic distribution	Application Load Balancer	Sends each origin request to a healthy API task in any Availability Zone.
Application logic	Amazon ECS on AWS Fargate	Runs the BookStash API and worker as containers without servers to manage.
System of record	Amazon RDS for PostgreSQL	Stores books, stock and orders durably with automated backups.
Hot data	Amazon ElastiCache	Holds frequently read book records in memory for sub-millisecond reads.
Deferred work	Amazon SQS	Buffers orders so the API can reply at once and the worker can catch up.
Visibility	CloudWatch and X-Ray	Metrics, logs, alarms and per-request traces across all tiers.

1.1.3 The Request and Response Round Trip

To make the system concrete we follow one request: a reader in Hyderabad opens the detail page for book 42. The browser issues GET /api/books/42. Figure 1.2 shows an illustrative timeline for the case where neither CloudFront nor ElastiCache holds the record, which is the slowest normal path. The numbers are typical of what we measure in the lab at the end of this chapter, but they vary with network conditions and should be read as orders of magnitude.

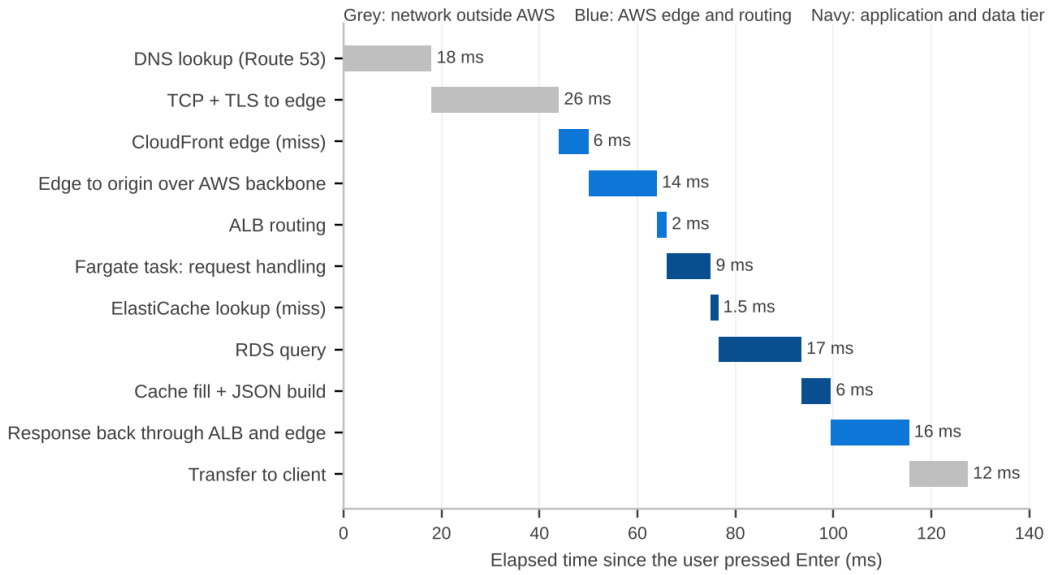


Figure 1.2 Illustrative latency waterfall for a cold GET /api/books/42 from Hyderabad. Values are representative, not guarantees.

Three observations from the waterfall shape the rest of the course. First, a large share of the total time is spent before the request reaches our code at all: name resolution and the TCP and TLS handshakes account for roughly a third. This is why CDNs and connection reuse matter so much. Second, inside the application tier the database query is the single largest item, which is why we cache. Third, the network legs back to the reader are not free; a response that is smaller or compressed arrives sooner.

EXAMPLE 1.1 Building a latency budget for BookStash

The product owner wants the book detail API to respond within 200 ms for 95 percent of requests made from India. We write down a budget per hop and compare it with what we measured.

Table 1.3 Latency budget versus measurement for GET /api/books/42 (milliseconds)

Hop	Budget	Cold path	Warm path
DNS and connection setup	50	44	0 (reused)
Edge processing and transfer to origin	30	20	20
ALB and task handling	15	11	11
Cache and database	60	18.5	1.5
Response build and return	45	34	28
Total	200	127.5	60.5

Example 1.1 (continued)

The cold path fits inside the budget with room to spare, and the warm path, where the browser reuses its connection and ElastiCache holds the record, is about twice as fast. The budget also tells us where to look if a release pushes the 95th percentile above 200 ms: the cache and database line has the largest allowance and the largest variance.

1.1.4 Why We Observe the Whole Before the Parts

There is a strong temptation, when learning cloud computing, to go service by service: one week on compute, one week on storage, one week on databases. That approach produces engineers who can configure each service correctly but cannot explain why the system as a whole is slow, expensive or fragile. Problems in production almost never sit inside one box. They sit on the arrows between boxes: a timeout that is shorter than a database failover, a cache key that is never invalidated, a health check that passes while the dependency it needs is down.

Seeing the whole system first gives us three habits that we use for the rest of the course.

- **Follow the request.** For any symptom we ask which hop the request was on when time was lost or an error appeared.
- **Know the boundary of each component.** Each box has an owner, a failure mode and a limit. The ALB does not know whether the database is healthy unless our health check tells it.
- **Measure before changing.** Every claim about performance or reliability in this book is backed by a metric or a trace that the reader can reproduce.

NOTE Where the code lives

The BookStash source, its container images and the infrastructure templates are provided in the course repository. In this chapter we only observe the deployed system. How the images are built and promoted is covered in the CICD course, and how we would redesign BookStash for millions of users is covered in the SDSR course. Section 1.8 explains these boundaries.

1.1.5 Observing the Running Application

We can learn a surprising amount about BookStash with nothing more than a terminal. The commands below are the first exercises we run in the lab. We show them here with trimmed output so that the reader can recognise each layer from the outside.

```
$ dig +short bookstash.com
13.224.163.12
13.224.163.84
13.224.163.37
```

```
13.224.163.101

$ curl -sI https://bookstash.com/covers/42.jpg | grep -iE "x-cache|x-amz-cf-pop|age|cache-control"
cache-control: public, max-age=86400
age: 5123
x-cache: Hit from cloudfront
x-amz-cf-pop: HYD50-P1

$ curl -s https://bookstash.com/api/books/42 -D - -o /dev/null | grep -iE "x-cache|x-amzn-trace-id"
x-cache: Miss from cloudfront
x-amzn-trace-id: Root=1-6740d8f2-3b9c0e1a7d2f4c5b6a8e9f01
```

The first command shows that bookstash.com resolves to several addresses, which belong to CloudFront rather than to any server we run. The second shows that the cover image was served from a cache in Hyderabad (the point of presence code begins with HYD), that it has been in that cache for 5,123 seconds, and that it may stay for a day. The third shows an API call that missed the edge cache and reached our origin. The ALB added a trace identifier to the request, and our API echoes it back in the response so that we can look it up. We will use that identifier in Section 1.6 to find the request in X-Ray.

1.2 Canonical Cloud-Native Architecture

BookStash was not designed from a blank page. It follows a pattern that appears, with small variations, in a very large share of web systems built on public clouds since the mid-2010s. We call it the canonical cloud-native architecture. The pattern is a stack of tiers, each with one job, connected by well-defined protocols. Figure 1.3 shows the tiers and the AWS services that fill them in this course.

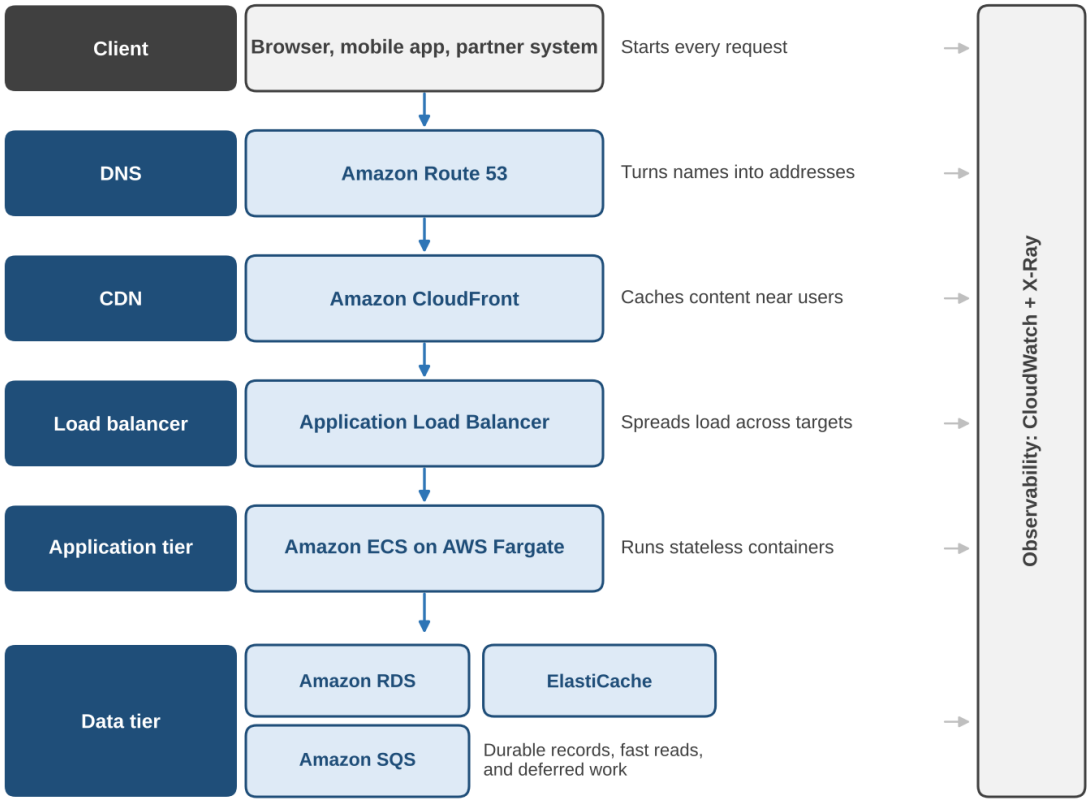


Figure 1.3 The canonical cloud-native architecture. Each tier has one responsibility; observability spans all tiers.

The word cloud-native deserves a short definition because it is used loosely. The Cloud Native Computing Foundation describes cloud-native technologies as those that let organisations build and run scalable applications in dynamic environments such as public, private and hybrid clouds, and it names containers, service meshes, microservices, immutable infrastructure and declarative APIs as typical examples. In this book we use a narrower working definition.

> **KEY CONCEPT Working definition: cloud-native application**
>
> An application is cloud-native when it is built from stateless processes that can be started, stopped and replaced at any time, when it keeps its durable state in managed data services rather than on local disks, when it is deployed and scaled through declarative configuration, and when it exposes the metrics, logs and traces needed to operate it without logging in to a machine.

BookStash meets every part of this definition. Its API tasks hold no state of their own, so ECS can replace one at any moment. Its data lives in RDS, ElastiCache and SQS. Its deployment is described in templates. Its behaviour is visible in CloudWatch and X-Ray. In the remainder of this section we take each tier in turn.

1.2.1 The Client

The client is anything that originates a request: a browser, a mobile application, a command-line tool, or another company's system calling our API. We do not control the client, and that single fact explains several design decisions. We cannot assume the client is close to our servers, so we use a CDN. We cannot assume the client retries sensibly, so our write operations must tolerate duplicates. We cannot assume the client's network is reliable, so responses should be small and connections should be reused.

For BookStash the main client is a single-page application loaded in the browser. It downloads its JavaScript bundle once, from CloudFront, and then calls the JSON API. A smaller number of requests come from a staff tool that updates stock.

1.2.2 DNS with Amazon Route 53

The Domain Name System (DNS) translates human-readable names into network addresses. It is the first dependency of every request and, because answers are cached at many levels, it is also the slowest part of the system to change. Figure 1.4 shows a full resolution for bookstash.com when no cache along the way has the answer.

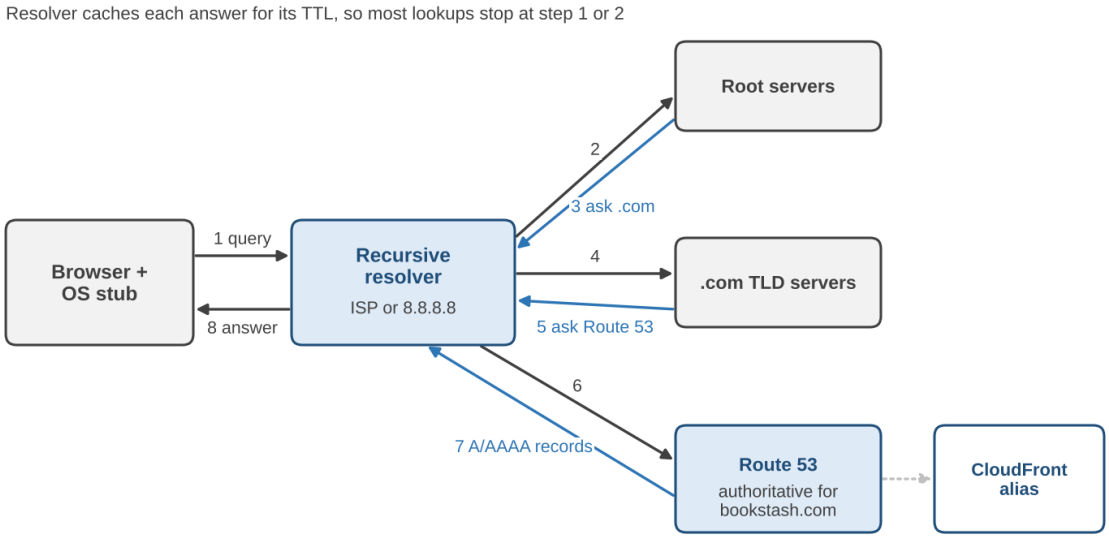


Figure 1.4 DNS resolution for bookstash.com. Route 53 is the authoritative name server; the recursive resolver does the walking.

Route 53 is AWS's managed DNS service. In BookStash it plays three roles. It is the **authoritative name server** for the bookstash.com hosted zone, which means it holds the definitive records. It provides **alias records**, an AWS extension that lets the apex name bookstash.com point directly at a CloudFront distribution without a CNAME, and alias queries to AWS resources carry no query charge. It also runs **health checks** and supports routing policies that change the answer based on health, latency, location or weight.

Table 1.4 Route 53 routing policies and when we use them

Policy	Behaviour and typical use
Simple	One answer for a name. Used for bookstash.com today.
Weighted	Splits traffic by percentage between answers. Used for gradual migrations and blue and green releases.
Latency-based	Answers with the Region that gives the reader the lowest measured latency. Used when the same stack runs in several Regions.
Failover	Returns a secondary answer when the primary fails its health check. Used for disaster recovery.
Geolocation and geoproximity	Answers based on the reader's country, continent or distance. Used for data residency and regional content.
Multivalue answer	Returns up to eight healthy records. A simple form of client-side load spreading.
IP-based	Answers based on the source network of the query. Used when traffic sources are known in advance.

EXAMPLE 1.2 Why a DNS change takes time to take effect

Suppose the bookstash.com record had a time to live (TTL) of 3,600 seconds and we change it at 10:00. A recursive resolver that fetched the old answer at 09:59 may keep serving it until 10:59. Browsers and operating systems may add their own caching on top. For planned changes we therefore lower the TTL to 60 seconds a day before the change, make the change, confirm that traffic has moved, and raise the TTL again. Alias records to AWS resources take their TTL from the target, which is one reason they are preferred for CloudFront and load balancers.

1.2.3 Content Delivery with Amazon CloudFront

A content delivery network (CDN) is a set of servers placed close to users that cache content and accept connections on behalf of an origin. CloudFront is AWS's CDN. Readers connect to the nearest of several hundred points of presence; CloudFront then either answers from its cache or fetches from the origin over the AWS backbone. Between the edge locations and the origins sit **regional edge caches**, larger caches that catch requests which miss at the edge. Figure 1.5 compares the two paths.

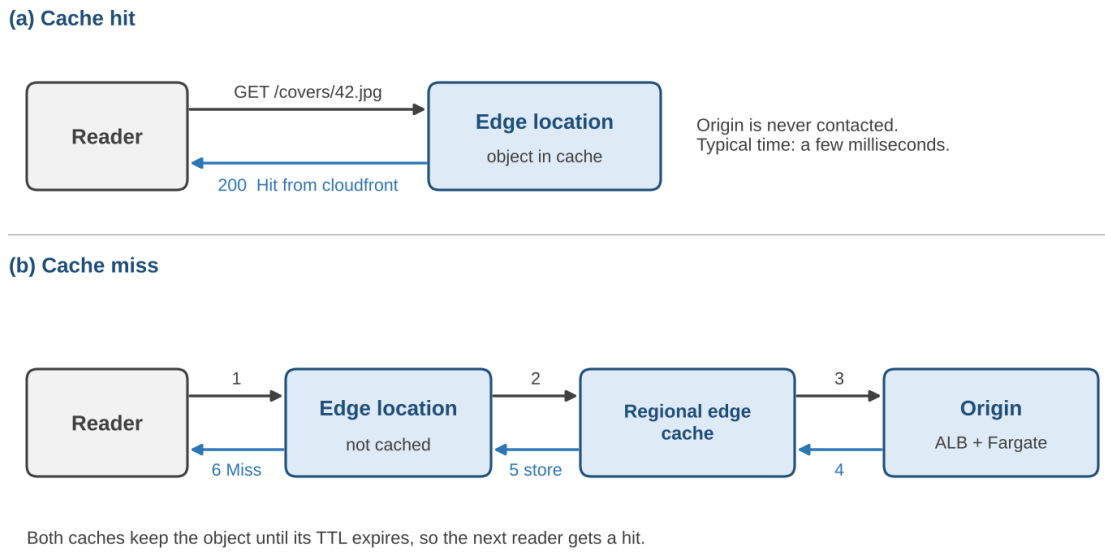


Figure 1.5 CloudFront request handling. (a) A cache hit is answered at the edge. (b) A miss travels through the regional edge cache to the origin and fills both caches on the way back.

CloudFront gives BookStash more than caching. Because readers terminate TLS at the edge, the expensive handshake happens over a short distance. Connections from the edge to the origin are long-lived and reused, so the origin sees fewer new connections. The distribution can also apply

AWS WAF rules, restrict access by country and absorb many network-layer attacks before they reach the VPC.

What CloudFront caches is controlled by **cache behaviours**, which match URL patterns, and by **cache policies**, which decide which headers, cookies and query strings form the cache key and how long objects live. Table 1.5 shows the behaviours configured for BookStash.

Table 1.5 CloudFront cache behaviours for BookStash

Path pattern	Cache TTL	Notes
/covers/*	1 day	Images never change for a given ID. Compressed, cached by path only.
/static/*	1 year	JavaScript and CSS bundles with content hashes in their names.
/api/books/*	60 seconds	Short TTL so stock changes appear quickly. Query strings excluded from key.
/api/search*	Not cached	Results depend on query and change often; forwarded to origin.
/api/orders	Not cached	POST requests are always forwarded.

EXAMPLE 1.3 What a cache hit ratio is worth

On a weekday BookStash serves about 1.2 million requests to /api/books/*. With a 60-second TTL CloudFront reports a cache hit ratio of 70 percent for that path. The origin therefore sees 0.3 x 1.2 million = 360,000 requests instead of 1.2 million. If one API task comfortably handles 150 requests per second at peak, and peak traffic is four times the daily average, we can estimate the tasks needed: average origin rate = 360,000 / 86,400 = 4.2 requests per second, peak = 16.7 requests per second, which one task handles easily. Without the CDN the peak would be 55.6 requests per second, still within one task but with far less headroom. We run at least two tasks anyway, for availability rather than for capacity.

1.2.4 The Application Load Balancer

A load balancer accepts connections on one address and spreads them across a pool of targets. The ALB operates at layer 7 of the OSI model: it understands HTTP, so it can route on path, host name, headers or query strings, and it can terminate TLS itself. In BookStash the ALB is the origin for CloudFront and the single entry point into the VPC.

An ALB has **listeners** (for example HTTPS on port 443), **rules** that map requests to **target groups**, and **health checks** that decide which targets receive traffic. Our target group holds the private IP addresses of the API tasks. Every 15 seconds the ALB calls /healthz on each task; two consecutive failures remove a task from rotation and three consecutive successes return it. Because the ALB itself has nodes in each Availability Zone, losing one zone does not take the entry point down.

> **NOTE Health checks that tell the truth**
>
> A health check that always returns 200 is worse than none, because it hides failures. Our /healthz handler checks that the task can reach ElastiCache and open a database connection from its pool within 100 ms. It does not run a full query, because a slow database should not cause the ALB to remove every task at once. Designing health checks that are neither too shallow nor too deep is a recurring theme in this course.

1.2.5 The Application Tier on Amazon ECS with AWS Fargate

The application tier runs our code. BookStash packages its API and its worker as container images. Amazon Elastic Container Service (ECS) is the orchestrator: it keeps the requested number of copies of each container running, replaces those that fail, registers them with the ALB and scales them. AWS Fargate is the compute engine underneath: it runs each **task** (one or more containers that are scheduled together) in its own isolated environment without exposing any server to us.

An ECS deployment is described by a **task definition** (image, CPU and memory, ports, environment, IAM role, logging) and a **service** (how many tasks, which subnets, which load balancer). The extract below is a trimmed version of the BookStash API task definition.

```
{
  "family": "bookstash-api",
  "requiresCompatibilities": ["FARGATE"],
  "networkMode": "awsvpc",
  "cpu": "512",
  "memory": "1024",
  "taskRoleArn": "arn:aws:iam::111122223333:role/bookstash-api-task",
  "containerDefinitions": [{
    "name": "api",
    "image": "111122223333.dkr.ecr.ap-south-1.amazonaws.com/bookstash-api:1.4.2",
    "portMappings": [{ "containerPort": 8080 }],
    "environment": [{ "name": "CACHE_TTL_SECONDS", "value": "300" }],
    "secrets": [{ "name": "DB_PASSWORD",
                  "valueFrom":   "arn:aws:secretsmanager:ap-south-1:111122223333:secret:bookstash-db"
}],
    "logConfiguration": { "logDriver": "awslogs",
      "options": { "awslogs-group": "/ecs/bookstash-api", "awslogs-region": "ap-south-1",
                   "awslogs-stream-prefix": "api" } }
  }]
}
```

Three details in this definition reflect the cloud-native principles from earlier. The task has its own IAM role, so it receives temporary credentials and never stores access keys. The database password is fetched from AWS Secrets Manager at start-up rather than baked into the image. Logs go straight to CloudWatch Logs, so nobody needs to log in to read them.

1.2.6 The Data Tier: Amazon RDS, Amazon ElastiCache and Amazon SQS

Stateless application tasks are only possible because state lives somewhere else. BookStash uses three managed data services, each chosen for a different access pattern.

Amazon RDS (Relational Database Service) runs PostgreSQL for us. AWS provisions the instance, applies minor engine patches in a maintenance window we choose, takes automated daily snapshots, retains transaction logs for point-in-time recovery and, in a Multi-AZ deployment, keeps a synchronous standby in a second Availability Zone that takes over automatically if the primary fails. RDS is the **system of record**: if a fact about a book or an order is not in RDS, it is not true.

Amazon ElastiCache runs an in-memory data store (Valkey or Redis OSS in our case) that answers simple key lookups in well under a millisecond. We use it with the **cache-aside** pattern shown in Figure 1.6(a): the API asks the cache first, falls back to the database on a miss and writes the result back with a TTL.

Amazon SQS (Simple Queue Service) is a fully managed message queue. The API places each order on the queue and returns 202 Accepted at once; the worker polls the queue, processes the order and deletes the message only when it has finished. If the worker crashes halfway, the message becomes visible again after its visibility timeout and another worker picks it up. Figure 1.6(b) shows this pattern.

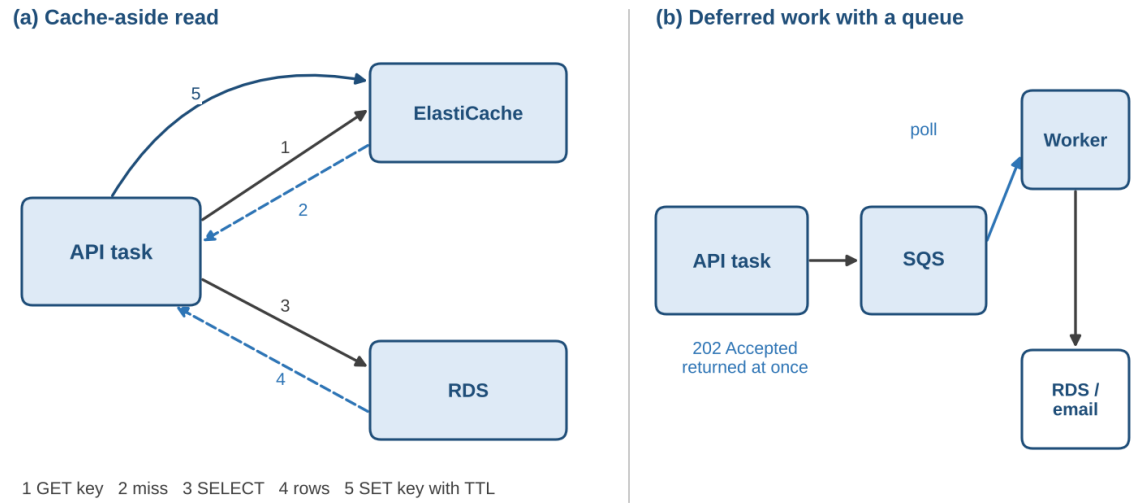


Figure 1.6 Two data-tier patterns in BookStash. (a) Cache-aside reads through ElastiCache. (b) Deferred order processing through SQS.

The cache-aside read in the API is short enough to show in full. The code is Python using the redis and psycopg libraries; error handling is reduced to the minimum needed to show the pattern.

```
import json, os
```

```
import redis, psycopg_pool

cache = redis.Redis(host=os.environ["CACHE_HOST"], port=6379, ssl=True)
pool = psycopg_pool.ConnectionPool(os.environ["DB_DSN"], min_size=2, max_size=10)
TTL = int(os.environ.get("CACHE_TTL_SECONDS", "300"))

def get_book(book_id: int) -> dict | None:
    key = f"book:{book_id}"
    cached = cache.get(key)
    if cached is not None:
        return json.loads(cached)
    with pool.connection() as conn:
        row = conn.execute(
            "SELECT id, title, author, price, stock FROM books WHERE id = %s",
            (book_id,)).fetchone()
    if row is None:
        return None
    book = dict(zip(("id", "title", "author", "price", "stock"), row))
    cache.set(key, json.dumps(book), ex=TTL)
    return book
```

Table 1.6 Comparison of the three BookStash data services

Aspect	Amazon RDS	Amazon ElastiCache	Amazon SQS
Role	System of record	Hot-data cache	Work buffer
Data model	Relational tables, SQL	Keys and simple structures	Messages
Typical latency	Single-digit to tens of ms	Under 1 ms	Tens of ms to enqueue
Durability	Multi-AZ, backups, PITR	Replicas; data may be lost and rebuilt	Stored redundantly until deleted or expired
Failure effect on BookStash	Reads and writes fail; orders queue up	Slower reads served from RDS	Orders cannot be accepted
Scaling lever	Instance size, read replicas	Node size, shards, serverless	Effectively unlimited throughput for standard queues

1.2.7 Observability with Amazon CloudWatch and AWS X-Ray

Observability is the ability to explain the internal state of a system from the signals it emits. In practice we rely on three signal types. **Metrics** are numbers over time, such as request count, latency percentiles or CPU utilisation; they tell us that something is wrong. **Logs** are timestamped records of events; they tell us what happened in one component. **Traces** connect the work done for one request across all components; they tell us where the time went.

CloudWatch collects metrics from every AWS service automatically (the ALB reports TargetResponseTime and HTTPCode_Target_5XX_Count, RDS reports CPUUtilization and DatabaseConnections, SQS reports ApproximateAgeOfOldestMessage, and so on), stores our application logs, evaluates alarms and draws dashboards. X-Ray stores and visualises traces. Since February 2026 AWS recommends instrumenting applications with OpenTelemetry, usually through the AWS Distro for OpenTelemetry (ADOT), and sending the resulting spans to X-Ray;

the older X-Ray SDKs and daemon are in maintenance mode. The concepts we teach (trace, segment or span, annotation, sampling) are the same either way.

Table 1.7 First CloudWatch alarms we define for BookStash

Metric	Threshold	Why it matters
ALB TargetResponseTime p95	Above 0.2 s for 5 minutes	Direct measure of the latency budget.
ALB HTTPCode_Target_5XX_Count	Above 1% of requests	Application errors seen by readers.
ECS service RunningTaskCount	Below 2	Loss of redundancy in the API tier.
RDS FreeStorageSpace	Below 20%	Running out of disk stops writes.
SQS ApproximateAgeOfOldestMessage	Above 300 s	Worker is not keeping up with orders.
ElastiCache CacheHitRate	Below 80% for 15 minutes	Cache is cold or keys are wrong; database load will rise.

CASE STUDY The canonical stack outside BookStash

A ticket booking platform for a regional bus operator in South India uses the same tiers with different emphasis. Its traffic is extremely spiky: when a festival weekend opens for booking, request rates jump by a factor of fifty within minutes. The team leans heavily on the CDN for the static seat-map application, sets a minimum of six API tasks during booking windows, and puts every seat-hold request through a FIFO queue grouped by bus and date so that two readers cannot hold the same seat. The database is the same managed PostgreSQL service we use. The lesson is that the canonical architecture is a starting point; the proportions and settings follow the workload.

1.3 Cloud Service Models

Every component in BookStash is a service that AWS operates on our behalf, but the services differ in how much they hide. When we run a container on Fargate we never see the server underneath. When we run an EC2 instance we see a full operating system and must patch it. When we use a hosted email product we see only a web page. The **service model** describes this difference: which layers of the stack the provider manages and which the customer manages.

1.3.1 The NIST Definition

The most widely cited definition of cloud computing comes from the United States National Institute of Standards and Technology, Special Publication 800-145. It describes cloud computing as a model for enabling convenient, on-demand network access to a shared pool of configurable computing resources that can be rapidly provisioned and released with minimal management effort or service provider interaction. It names five essential characteristics, three service models and four deployment models. Table 1.8 summarises them.

Table 1.8 Elements of the NIST cloud computing definition (SP 800-145)

Element	Meaning and BookStash example
On-demand self-service	We create resources through an API or console without a ticket to a human. We created the RDS instance in minutes.
Broad network access	Services are reachable over standard network protocols from many client types. Readers use browsers and phones.
Resource pooling	The provider serves many customers from shared hardware with logical isolation. Our Fargate tasks share physical hosts with other customers' workloads.
Rapid elasticity	Capacity can grow and shrink quickly, often automatically. ECS adds tasks when CPU stays above 60 percent.
Measured service	Usage is metered and billed. We pay per vCPU-second for Fargate and per request for SQS.
Service models	Infrastructure, Platform and Software as a Service (IaaS, PaaS, SaaS).
Deployment models	Private, community, public and hybrid cloud. BookStash runs in a public cloud.

The three original service models were defined in 2011. Since then the industry has added at least two more that are common enough to need names: **Function as a Service (FaaS)** and **Container as a Service (CaaS)**. We treat all five in this section. Figure 1.7 shows, layer by layer, who manages what under each model.

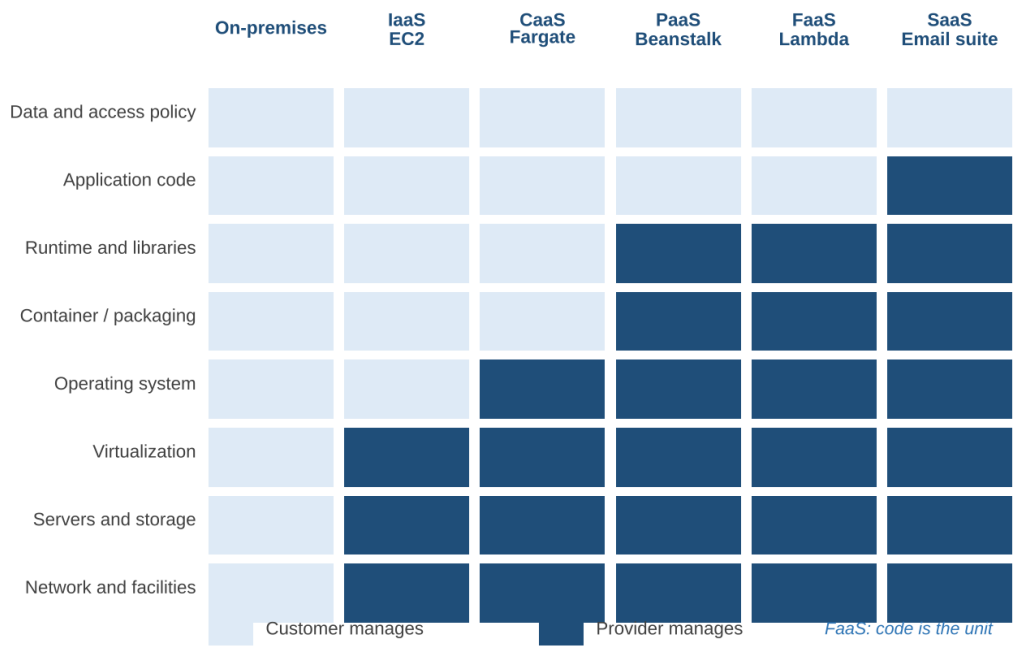


Figure 1.7 Management responsibility by service model. Light cells are managed by the customer, dark cells by the provider. The AWS service under each heading is an example, not the only one.

1.3.2 Infrastructure as a Service (IaaS)

IaaS provides virtual machines, block storage and virtual networks. The provider runs the data centre, the physical servers, the network fabric and the hypervisor. The customer chooses an operating system image and is responsible for everything from the operating system upwards: patching, hardening, installing runtimes, deploying code and scaling the number of machines.

On AWS the core IaaS services are Amazon EC2 (virtual machines, called instances), Amazon EBS (block volumes attached to instances) and Amazon VPC (private networks with subnets, route tables and gateways). Equivalent services exist on Azure (Virtual Machines, Managed Disks, Virtual Network) and Google Cloud (Compute Engine, Persistent Disk, VPC).

IaaS gives the most control. We can install any software, tune kernel parameters and attach specialised hardware such as GPUs. The price of that control is operational work. If BookStash ran its API on EC2, we would need to build an image, keep it patched, configure an Auto Scaling group across Availability Zones, install the CloudWatch agent to see memory and disk, and rotate instances when a new kernel fix is released.

EXAMPLE 1.4 Running the BookStash API on EC2 instead of Fargate

In an early version of the course we deployed BookStash on two t3.small EC2 instances behind the same ALB. The list of tasks that fell to us was noticeably longer than with Fargate:

- choose and maintain an Amazon Linux image, and apply security updates monthly;
- install Docker or a language runtime, the CloudWatch agent and a log shipper;
- size instances so that both CPU and memory have headroom, and pay for idle capacity at night;
- configure an Auto Scaling group with a launch template and health checks;
- plan replacement of instances during a kernel or runtime patch without dropping requests.

The application code was identical. The difference was entirely in the layers below it, which is exactly what the service model describes.

1.3.3 Platform as a Service (PaaS)

PaaS moves the boundary higher. The provider runs the operating system, the language runtime and often the web server, and the customer supplies application code and configuration. Scaling, patching and load balancing are handled by the platform within limits that the customer sets.

AWS Elastic Beanstalk is the classic AWS example: we upload a Python, Java, Node.js, .NET, Go, PHP or Ruby application and Beanstalk provisions instances, a load balancer and scaling for us, and applies managed platform updates. Azure App Service and Google App Engine play the same role. Managed databases such as Amazon RDS are often also described as PaaS, because the provider runs the database engine and the customer manages schemas, queries and data.

The benefit of PaaS is speed: a team can go from code to a running, scalable service in an afternoon. The cost is reduced flexibility. The platform decides which runtime versions are available, how the file system behaves and how requests are routed. Applications that need unusual system libraries or long-running background processes can be awkward to fit.

1.3.4 Software as a Service (SaaS)

SaaS delivers a complete application over the network. The customer configures and uses it but does not build or operate it. Examples include email and productivity suites, customer relationship management systems, payroll systems and source code hosting services. The provider is responsible for the whole stack, including the application code. The customer remains responsible for its own data, for who has access, and for how the product is configured.

BookStash itself could be offered as SaaS: other universities could subscribe to it rather than run their own copy. That shift would bring new responsibilities for us as the provider, such as tenant isolation, per-tenant billing and service-level commitments, which we touch on in Section 1.4.

1.3.5 Function as a Service (FaaS)

FaaS runs individual functions in response to events. The customer writes a function and declares what triggers it (an HTTP request, a message on a queue, a file upload, a schedule). The platform starts an execution environment when an event arrives, runs the function, and scales the number of concurrent environments with the event rate, including down to zero when there are no events. Billing is per invocation and per unit of execution time.

AWS Lambda is the main AWS FaaS service. Azure Functions and Google Cloud Run functions (formerly Cloud Functions) are the equivalents. FaaS is often grouped under the broader label **serverless**, which also covers services such as SQS, DynamoDB and Fargate that require no server management and scale automatically.

In BookStash a natural FaaS candidate is the generation of cover thumbnails. When staff upload a new cover image, a Lambda function triggered by the upload could produce three sizes and store them. The work is short, event-driven and irregular, which matches the FaaS model well.

> **NOTE When FaaS is not the right fit**
>
> FaaS functions have a maximum execution time (15 minutes for Lambda), start new environments on demand (which can add latency to the first request, known as a cold start), and keep no local state between invocations. Workloads that run continuously at high, steady load, that hold long-lived connections, or that need large amounts of memory for a long time are usually cheaper and simpler on containers. We chose containers for the BookStash API for this reason and keep FaaS for event-driven side tasks.

1.3.6 Container as a Service (CaaS)

CaaS sits between IaaS and PaaS. The unit of deployment is a container image, which the customer builds and owns, including its operating system user space, runtime and libraries. The provider runs the container orchestration and, in the serverless variants, the hosts as well.

On AWS there are two orchestrators, Amazon ECS and Amazon Elastic Kubernetes Service (EKS), and two ways to supply compute: EC2 instances that the customer manages, or AWS Fargate, where AWS manages the hosts. BookStash uses ECS on Fargate, which is the most managed of the four combinations. Azure Container Apps and Azure Kubernetes Service, and Google Cloud Run and Google Kubernetes Engine, are the equivalents.

CaaS has become the default for new web services because the container image is portable (the same image runs on a laptop, in a test environment and in production), because it gives the customer full control of the runtime without control of the hosts, and because orchestrators handle health, placement and scaling.

1.3.7 Abstraction Versus Control

It is tempting to rank the models from worst to best, with SaaS at the top. That is a mistake. Each step up the abstraction ladder trades control for reduced operational work, and the right point depends on the workload and the team. Figure 1.8 places the AWS compute options on a single spectrum.

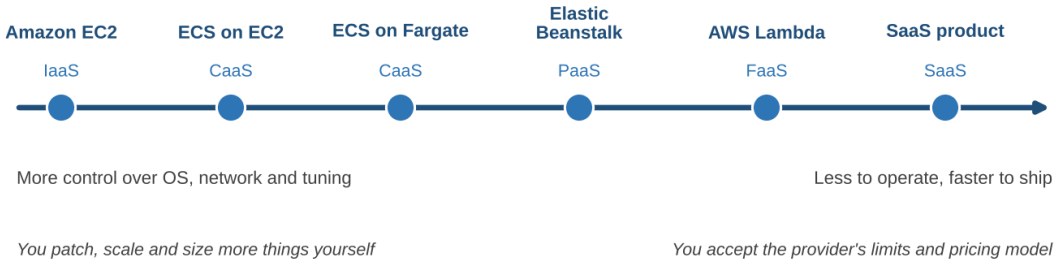


Figure 1.8 The abstraction and control spectrum for compute on AWS.

Table 1.9 Comparison of cloud service models

Aspect	IaaS	CaaS	PaaS	FaaS	SaaS
Unit you deploy	VM image	Container image	Application code	Function code	Configuration only
You manage OS	Yes	No (on Fargate)	No	No	No
Scaling	You configure groups	Orchestrator scales tasks	Platform scales	Per event, to zero	Provider
Billing basis	Instance-hours	vCPU and memory per second	Instances or plan	Requests and duration	Per user or tier
Startup time	Minutes	Seconds to a minute	Seconds to minutes	Milliseconds to seconds	None
Portability	High	High	Medium	Low to medium	Low
Example	Amazon EC2	ECS or EKS on Fargate	Elastic Beanstalk	AWS Lambda	Microsoft 365
BookStash use	Not used	API and worker	Not used	Thumbnail generator	Staff email

EXAMPLE 1.5 Choosing a service model for three BookStash features

We apply a simple set of questions to three features: Does it run continuously or only on events? Do we need a custom runtime? How much operational effort can the team spare?

Table 1.10 Service model choices for three BookStash features

Feature	Load pattern	Choice	Reason
Book API	Continuous, steady with daily peaks	CaaS (ECS on Fargate)	Long-running, needs connection pools, custom libraries.
Cover thumbnails	Bursts when staff upload	FaaS (Lambda)	Short, event-driven, idle most of the day.
Monthly sales report	Once a month, 20 minutes	FaaS or scheduled Fargate task	Lambda if it fits in 15 minutes; otherwise a scheduled task.

1.3.8 Service Abstraction and Customer Responsibility

The key idea to take from this section is that moving up the abstraction ladder does not remove customer responsibility; it changes its shape. When we move from EC2 to Fargate we stop patching operating systems, but we still choose the container base image and must rebuild it when a library vulnerability is announced. When we move from a self-managed PostgreSQL to RDS we stop installing engine patches, but we still design indexes, size the instance, decide on backup retention and control who can connect. When we use SaaS we stop writing code, but we still decide who has accounts and what data goes in.

This observation leads directly to the shared responsibility model.

1.4 The Shared Responsibility Model

Security and reliability in the cloud are jointly delivered. AWS is responsible for some things, the customer for others, and the dividing line moves depending on the service. AWS describes this as the **shared responsibility model** and summarises it in a short phrase: AWS is responsible for security **of** the cloud, and the customer is responsible for security **in** the cloud. Figure 1.9 shows the model.

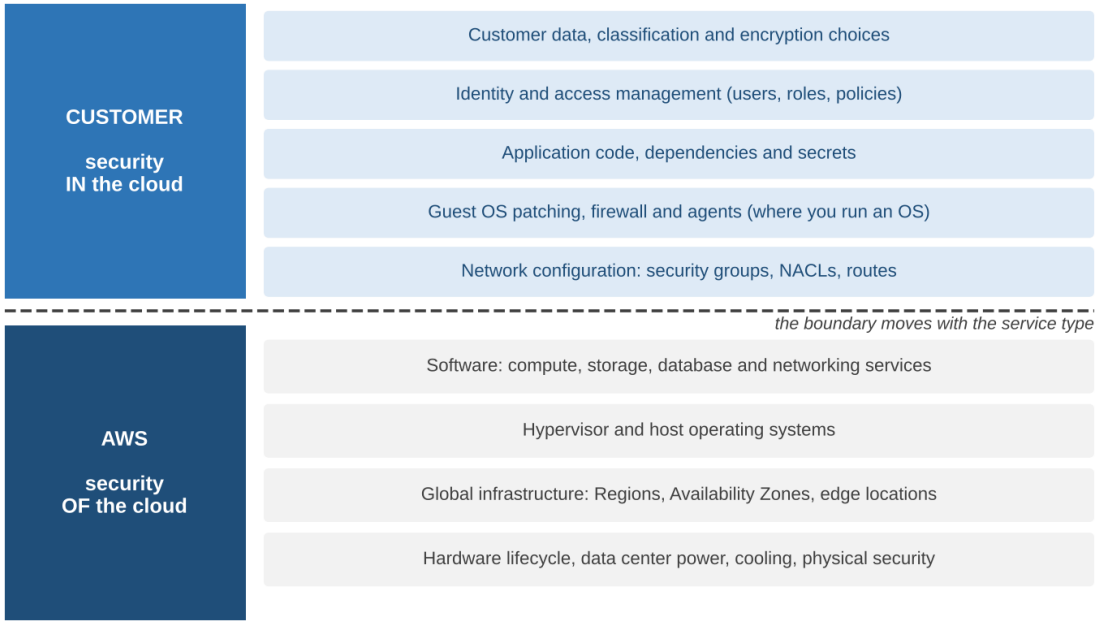


Figure 1.9 The AWS shared responsibility model. The lower block is always AWS's; how much of the upper block the customer carries depends on the service.

1.4.1 AWS Responsibilities and Guarantees

AWS is responsible for protecting the infrastructure that runs all of its services. That includes the physical security of data centres (access control, surveillance, staff vetting), the hardware lifecycle (including secure destruction of storage media), power and cooling, the global network, the hypervisor that isolates one customer's virtual machines from another's, and the software that implements managed services such as RDS, SQS and ElastiCache.

Customers cannot inspect most of this directly, so AWS demonstrates it through third-party audits and certifications. Reports for SOC 1, SOC 2 and SOC 3, ISO/IEC 27001 and related standards, PCI DSS and many regional schemes are available to customers through AWS Artifact. For an Indian public sector or banking customer, these reports form part of the evidence an auditor will ask for.

AWS also makes **service commitments** in the form of Service Level Agreements (SLAs). An SLA states a monthly uptime percentage for a service and the service credit the customer receives if it is missed. An SLA is a commercial commitment, not an engineering guarantee: it tells us what AWS will refund, not that the service will never fail. We design BookStash on the assumption that any single component can be unavailable for some minutes.

1.4.2 Customer Responsibilities

The customer is responsible for everything it puts into the cloud and for how it configures the services it uses. For BookStash, our responsibilities include the following.

- **Identity and access management.** Who in our team can change the production account; what each ECS task role is allowed to do; whether multi-factor authentication is enforced.
- **Data protection.** Whether RDS storage, snapshots and the SQS queue are encrypted; which keys are used; how long backups are retained; whether personal data of readers is minimised.
- **Network configuration.** Which security groups allow traffic to the database (only from API and worker tasks); that the RDS instance is not publicly accessible; that the ALB accepts traffic only from CloudFront.
- **Application security.** Input validation, dependency updates, protection against injection and broken authentication, secrets kept in Secrets Manager.
- **Operating system and runtime**, where we run them. On Fargate the host is AWS's, but the container image is ours.
- **Configuration of resilience features.** Multi-AZ for RDS, at least two tasks across zones, alarms, backups and tested restores.

> **KEY CONCEPT A simple test for ownership**
>
> If a setting can be changed from our AWS account, the consequences of that setting are our responsibility. AWS provides encryption for RDS; we decide whether to turn it on. AWS provides Multi-AZ; we decide whether to pay for it. AWS provides security groups; we decide what they allow.

1.4.3 How the Boundary Moves with the Service

Because the service models hide different layers, the shared responsibility line sits in a different place for each service. Figure 1.10 shows this for five services that BookStash uses or could use.

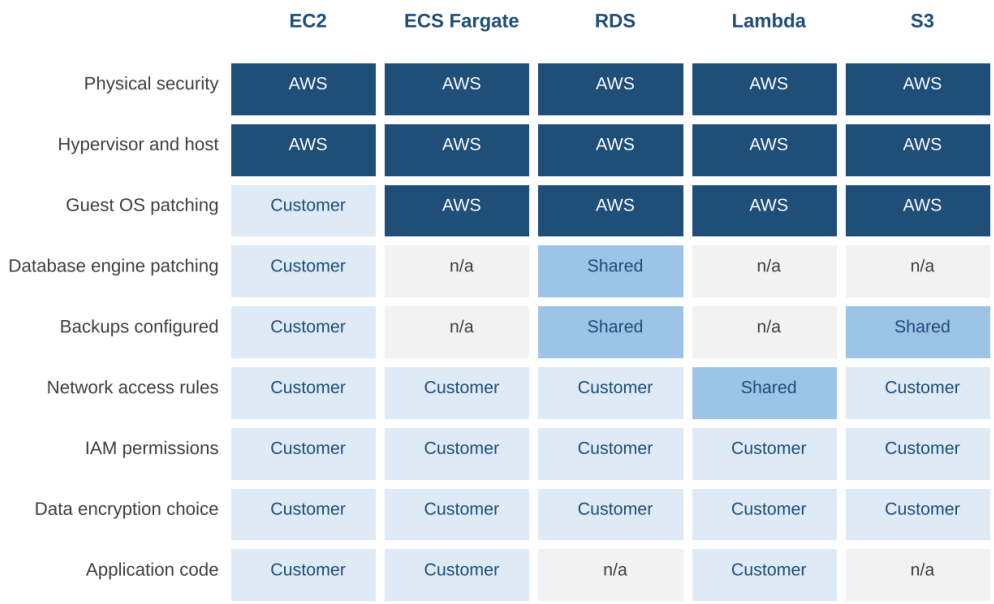


Figure 1.10 Who is responsible for common tasks across five AWS services. Shared cells mean AWS supplies the mechanism and the customer configures and verifies it.

The pattern is clear. Physical and host-level security is always AWS's. Identity, data classification and encryption choices are always the customer's. Everything in between depends on the service. For a self-managed database on EC2, the customer patches the operating system and the database engine. For RDS, AWS patches both, but the customer chooses the maintenance window and decides when to accept a major version upgrade. For S3, there is no operating system or engine to patch, but the customer must still configure bucket policies, Block Public Access and versioning.

1.4.4 Cloud Security Implications

Most security incidents in public clouds are not failures of the provider's infrastructure. They are configuration errors on the customer side of the line: a storage bucket left open to the internet, access keys committed to a public repository, a database security group that allows 0.0.0.0/0, an over-broad IAM policy that lets one compromised component reach everything. The shared responsibility model explains why: those settings are the customer's.

For BookStash we follow a short list of rules from the first day, and we revisit each of them in the security chapter.

Table 1.11 Baseline security rules for BookStash and the risk each addresses

Rule	Risk addressed
No long-lived access keys; tasks use IAM roles	Leaked keys giving permanent access to the account.
Least-privilege task roles (API can send to SQS but not delete the queue)	One compromised task reaching unrelated resources.
RDS and ElastiCache in private subnets; security groups reference task groups	Direct access to data stores from the internet.
Encryption at rest with AWS KMS for RDS, ElastiCache, SQS and logs	Exposure of data if storage media or snapshots are mishandled.
TLS on every hop: reader to CloudFront, CloudFront to ALB, task to RDS and cache	Interception of data in transit.
AWS CloudTrail enabled in all Regions with logs in a separate account	Unrecorded or deleted evidence of account activity.

1.4.5 Cloud Reliability Implications

AWS applies the same idea to resilience. In its Well-Architected guidance AWS describes a shared responsibility model for resiliency: AWS is responsible for the resiliency **of** the cloud (the infrastructure, Availability Zones and service implementations), and the customer is responsible for resiliency **in** the cloud (how the workload uses those services, how data is backed up, how failures are detected and recovered).

In concrete terms, AWS makes sure that each Availability Zone is independent and that services like ALB and SQS run across zones. It is our job to place BookStash tasks in more than one zone, to enable Multi-AZ on RDS, to set timeouts and retries sensibly and to test that a restore from backup actually works.

EXAMPLE 1.6 Estimating the availability of a chain of components

When a request must pass through several components in series, the availability of the chain is roughly the product of the individual availabilities, assuming independent failures. Suppose, for illustration, that the ALB is available 99.99 percent of the time, the API tier 99.95 percent, ElastiCache 99.9 percent and RDS 99.95 percent. A request that needs all four succeeds with probability:

```
A = 0.9999 x 0.9995 x 0.999 x 0.9995 = 0.9979   (about 99.79 %)
Downtime per 30-day month = (1 - 0.9979) x 30 x 24 x 60 = about 91 minutes
```

Two design choices improve this. First, the cache is optional for correctness: if ElastiCache fails, the API reads from RDS, so the cache drops out of the chain for availability and A rises to about 99.89 percent. Second, running two independent API tasks in different zones makes that tier fail only if both fail, so its availability becomes 1 - (1 - 0.9995)^2, which is close to 1. The example shows why we look at the shape of the dependency chain, not only at each component.

Table 1.12 Responsibility summary for BookStash components

Component	AWS is responsible for	We are responsible for
Route 53	Global name server fleet, query answering, health-check infrastructure	Records, TTLs, routing policy, domain registration renewal
CloudFront	Edge locations, TLS termination, DDoS absorption at edge	Behaviours, cache keys, certificates, WAF rules, origin protection
ALB	Load balancer nodes, scaling, multi-AZ placement	Listeners, rules, health checks, TLS policy, security groups
ECS on Fargate	Hosts, kernel, container runtime, task isolation	Images, task roles, CPU and memory sizing, task count, scaling policy
RDS	Hardware, OS, engine patching, Multi-AZ failover mechanism	Schema, queries, instance class, backup retention, encryption choice, access
ElastiCache	Nodes, engine, failover mechanism	Key design, TTLs, eviction policy, node size, encryption in transit
SQS	Storage of messages, delivery, scaling	Visibility timeout, dead-letter queue, idempotent consumers, access policy
CloudWatch and X-Ray	Collection, storage and query infrastructure	What to log and trace, alarms, retention, sampling, dashboards

1.5 AWS, Azure and GCP Compared

This course uses AWS as its primary platform. We made that choice because AWS has the broadest set of services, the largest share of the infrastructure market, and the most job openings that ask for it in India and abroad. The concepts, however, are not specific to AWS. Microsoft Azure and Google Cloud Platform (GCP) offer services for every tier of the canonical architecture, and an engineer who understands BookStash on AWS can build it on either of the others with a few days of adjustment. This section gives the vocabulary needed to make that move.

1.5.1 Why AWS Is the Primary Platform in This Course

A course needs one platform to make labs concrete. Splitting attention evenly across three providers would triple the amount of console navigation, command syntax and billing detail students have to absorb, while adding little conceptual depth. We therefore build and operate BookStash on AWS, and in each chapter we include a short mapping to Azure and GCP so that the ideas transfer.

There are three further practical reasons. AWS offers a free tier and educational credit programmes that cover the labs in this book. The AWS Certified Cloud Practitioner and Solutions Architect Associate certifications map closely onto the first half of the course. And many AWS service names have become generic terms in industry conversation (people say S3-compatible storage or Lambda-style functions even when using other providers), so learning them first helps with reading documentation and job descriptions.

1.5.2 Mapping BookStash to Azure and GCP

Figure 1.11 places the BookStash components side by side for the three providers. Each column is one tier of the canonical architecture.

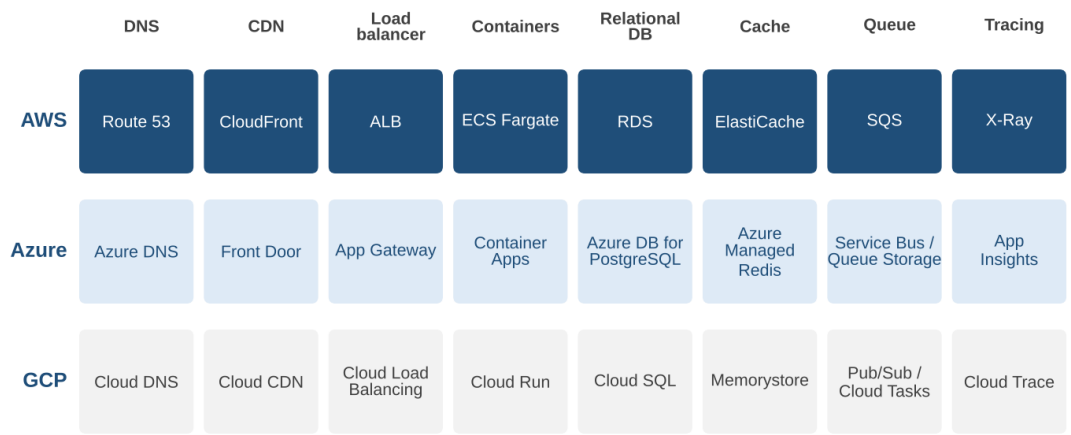


Figure 1.11 BookStash components on AWS, Azure and GCP.

The mapping is close but not exact. A few differences matter when we port a design.

- **Load balancing and CDN boundaries differ.** Azure Front Door is both a global CDN and a global layer-7 load balancer with web application firewall features. On AWS those roles are split between CloudFront and the ALB. GCP's global external Application Load Balancer is integrated with Cloud CDN, so caching is a setting on the load balancer rather than a separate service.
- **Serverless containers differ in shape.** Google Cloud Run and Azure Container Apps scale per request and can scale to zero. ECS on Fargate scales per task and keeps a minimum task count running. For BookStash, where we want two tasks warm at all times, the difference is small; for an internal tool used once a week it is significant.
- **Networks differ in scope.** An AWS VPC and an Azure Virtual Network belong to one Region. A GCP VPC is global, with subnets that are regional. A single GCP network can therefore span continents without peering.
- **Queues differ in semantics.** SQS and Azure Queue Storage are simple queues. Azure Service Bus adds topics, sessions and transactions. GCP Pub/Sub is a publish and subscribe system in which each subscription receives every message, and Cloud Tasks is closer to a task queue with explicit dispatch to an HTTP target.

1.5.3 Service Equivalence Tables

The tables in this subsection are the reference we return to throughout the course. They list the closest equivalent, not a perfect match; product boundaries, limits and pricing always differ in detail.

Table 1.13 Edge, network and compute equivalents

Capability	AWS	Microsoft Azure	Google Cloud
Authoritative DNS	Route 53	Azure DNS (Traffic Manager for routing policies)	Cloud DNS
Content delivery network	CloudFront	Azure Front Door	Cloud CDN
Layer-7 load balancer	Application Load Balancer	Application Gateway; Front Door for global	Application Load Balancer (Cloud Load Balancing)
Layer-4 load balancer	Network Load Balancer	Azure Load Balancer	Network Load Balancer
Private network	Amazon VPC	Virtual Network (VNet)	VPC network (global)
Virtual machines	Amazon EC2	Azure Virtual Machines	Compute Engine
Serverless containers	ECS on AWS Fargate	Azure Container Apps	Cloud Run
Managed Kubernetes	Amazon EKS	Azure Kubernetes Service (AKS)	Google Kubernetes Engine (GKE)
Functions	AWS Lambda	Azure Functions	Cloud Run functions
Platform as a Service	AWS Elastic Beanstalk	Azure App Service	App Engine
Container registry	Amazon ECR	Azure Container Registry	Artifact Registry

Table 1.14 Data, messaging and observability equivalents

Capability	AWS	Microsoft Azure	Google Cloud
Managed relational database	Amazon RDS, Amazon Aurora	Azure Database for PostgreSQL or MySQL, Azure SQL Database	Cloud SQL, AlloyDB
In-memory cache	Amazon ElastiCache	Azure Managed Redis	Memorystore
Message queue	Amazon SQS	Queue Storage, Service Bus	Pub/Sub, Cloud Tasks
Object storage	Amazon S3	Azure Blob Storage	Cloud Storage
NoSQL key-value and document	Amazon DynamoDB	Azure Cosmos DB	Firestore, Bigtable
Metrics, logs and alarms	Amazon CloudWatch	Azure Monitor, Log Analytics	Cloud Monitoring, Cloud Logging
Distributed tracing	AWS X-Ray	Application Insights	Cloud Trace

Table 1.15 Identity, security and management equivalents

Capability	AWS	Microsoft Azure	Google Cloud
Billing and isolation unit	Account	Subscription and resource group	Project
Hierarchy of units	AWS Organizations, organizational units	Management groups	Organization and folders

Capability	AWS	Microsoft Azure	Google Cloud
Identity and permissions	AWS IAM, IAM Identity Center	Microsoft Entra ID, Azure RBAC	Cloud IAM
Secrets storage	AWS Secrets Manager	Azure Key Vault	Secret Manager
Key management	AWS KMS	Azure Key Vault	Cloud KMS
Audit log of API calls	AWS CloudTrail	Azure Activity Log	Cloud Audit Logs
Infrastructure as code (native)	AWS CloudFormation, AWS CDK	ARM templates, Bicep	Infrastructure Manager (Terraform)
Edge firewall	AWS WAF	Azure Web Application Firewall	Cloud Armor

1.5.4 Infrastructure Vocabulary Across Providers

The three providers use similar words for their physical structures, with small differences that cause confusion in design reviews. Table 1.16 compares them. We study the AWS hierarchy in detail in Section 1.7.

Table 1.16 Geographic terms across providers

Concept	AWS	Microsoft Azure	Google Cloud
Geographic area with isolated infrastructure	Region	Region	Region
Isolated location within a Region	Availability Zone	Availability zone (not in every Region)	Zone
Paired location for disaster recovery	Not formalised; customer chooses	Region pair	Not formalised; dual- and multi-regions for storage
Points of presence for caching	Edge locations, regional edge caches	Front Door points of presence	Cloud CDN edge caches
Small extension near a metro area	Local Zone	Extended zone	Not a direct equivalent
Provider hardware on customer premises	AWS Outposts	Azure Local	Google Distributed Cloud

EXAMPLE 1.7 Porting BookStash to Google Cloud in one page

A student team ported BookStash to GCP as a course project. Their change list was short. The container images were unchanged. Cloud DNS replaced Route 53 with a managed zone. A global external Application Load Balancer with Cloud CDN enabled replaced both CloudFront and the ALB. Cloud Run replaced ECS on Fargate, with minimum instances set to two. Cloud SQL for PostgreSQL replaced RDS, with high availability enabled for a standby in a second zone. Memorystore replaced ElastiCache. Cloud Tasks replaced SQS for orders, which required a small code change because Cloud Tasks pushes work to an HTTP endpoint rather than letting a worker poll. Cloud Trace received the same OpenTelemetry spans that X-Ray received on AWS. The only part that took more than a day was rewriting the infrastructure templates.

NOTE Avoiding unnecessary lock-in

Every managed service creates some dependency on its provider. We do not try to avoid that entirely, because the operational benefit of managed services is the reason to use a cloud. We do keep the dependency visible and deliberate: standard protocols at the edges (HTTP, SQL, the Redis protocol), container images rather than provider-specific packaging for the main application, OpenTelemetry rather than a provider SDK for tracing, and a thin adapter around the queue client so that it can be swapped.

1.6 End-to-End Request Tracing

In Section 1.1 we watched a request from the outside. In this section we follow the same request from the inside, hop by hop, and we look at the evidence each component leaves behind. This skill is the most practical one in the chapter. When a reader complains that BookStash is slow, or when an alarm fires at two in the morning, we answer it by tracing requests.

1.6.1 The Path of One Request

We trace GET /api/books/42 when the record is not in any cache. Figure 1.12 shows the full exchange as a sequence diagram. Time flows from top to bottom; solid arrows are requests and dashed arrows are responses.

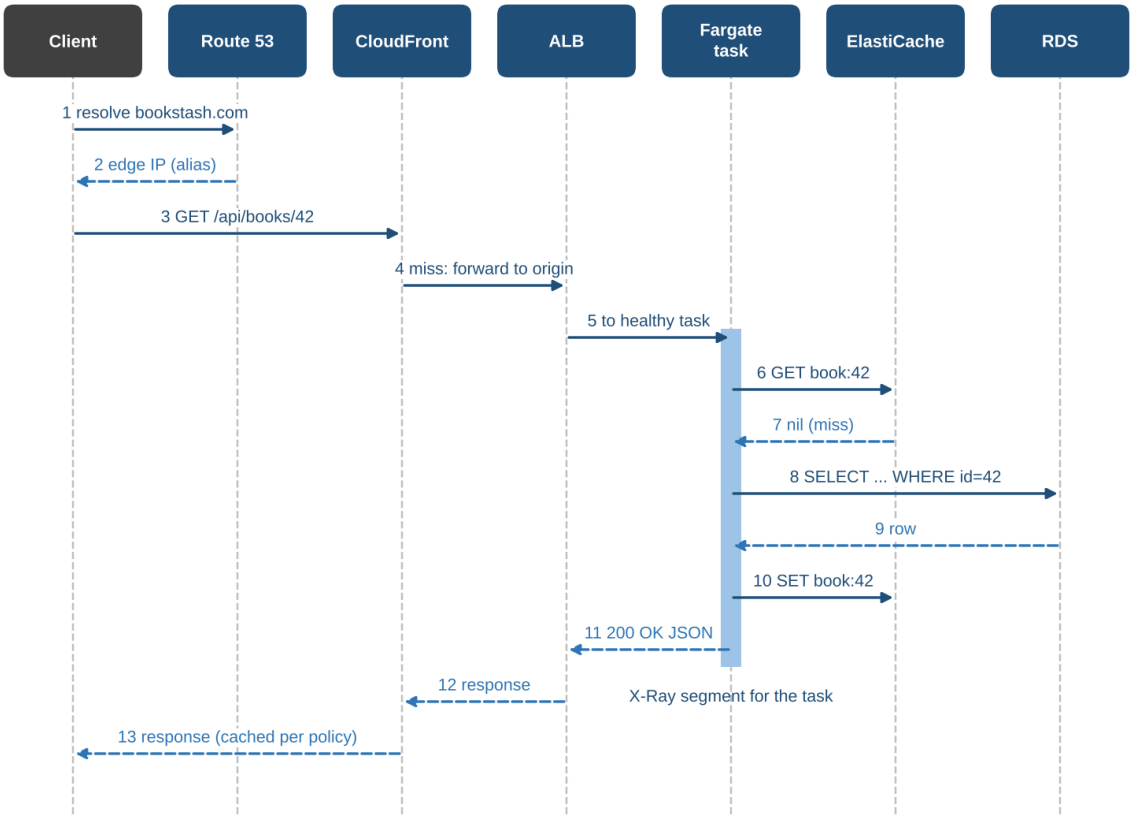


Figure 1.12 Sequence of messages for GET /api/books/42 on a cold path. The shaded bar on the Fargate lifeline is the part of the request recorded by X-Ray.

We now walk through each hop, noting what happens, how long it typically takes and where the evidence is recorded.

Hop 1: Route 53 resolves the name

The browser needs an IP address for bookstash.com. If the operating system or the recursive resolver has a cached answer, this hop costs nothing. If not, the resolver queries Route 53, which returns the addresses of nearby CloudFront edge locations (steps 1 and 2 in the figure). Route 53 itself does not take part in X-Ray traces. If we need evidence of DNS behaviour we enable **Route 53 query logging** for the hosted zone, which records each query that reached Route 53 with its name, type, response code and the edge location that answered.

Hop 2: CloudFront receives the request and misses

The browser connects to the edge location, completes the TLS handshake and sends the request (step 3). CloudFront computes the cache key from the path, finds no fresh object and forwards the request to the origin, which is our ALB (step 4). The edge adds headers that identify itself and the original client. CloudFront does not emit X-Ray segments either. Its evidence is in the **CloudFront access logs**, which record, for every request, the edge location, the result type (Hit, Miss, RefreshHit, Error and others), the time taken and a unique request identifier, and in CloudWatch metrics such as CacheHitRate and OriginLatency.

Hop 3: The ALB selects a task

The ALB terminates the TLS connection from CloudFront, evaluates its listener rules, picks a healthy target in the API target group and forwards the request (step 5). Before forwarding it adds an **X-Amzn-Trace-Id** header, or extends one that already exists. This header is the thread that ties the rest of the trace together. The ALB does not send its own segment to X-Ray, but its **access logs** include the same trace identifier together with request processing time, target processing time and response processing time, so we can see exactly how long the ALB waited for our task.

Hop 4: The Fargate task handles the request

The API container receives the request. Our OpenTelemetry instrumentation reads the trace header, starts a **segment** for the request and records each downstream call as a **subsegment** (OpenTelemetry calls both of these spans). The task asks ElastiCache for book:42 (step 6) and receives nothing (step 7). It queries RDS (steps 8 and 9), writes the result back to the cache (step 10), sends a small book-viewed event to SQS for the popularity ranking, serialises the JSON and responds (step 11).

Hops 5 and 6: ElastiCache and RDS

The cache lookup and the database query appear as subsegments with their own timings, the command or SQL statement (without parameter values) and any error. Neither service sends spans of its own; their side of the story is in their CloudWatch metrics (cache latency and hit rate, database CPU, connections and read latency) and, for RDS, in **Performance Insights**, which shows which queries consumed database time.

Hop 7: The response returns

The ALB passes the response back to CloudFront (step 12). CloudFront applies its cache policy, stores the object for 60 seconds because the path matches /api/books/*, and returns it to the browser (step 13). The next reader who asks for book 42 within the minute from the same edge is served without touching the origin.

Table 1.17 Where each hop leaves evidence

Hop	Primary evidence	What it tells us
Route 53	Query logs, health check status	Which name was asked, from which resolver, with what answer.
CloudFront	Access logs, CacheHitRate, OriginLatency	Hit or miss, edge location, total time at edge, origin wait.
ALB	Access logs with trace_id, TargetResponseTime	Time spent waiting for the task; which task; status codes.
Fargate task	X-Ray segment, application logs	Time in our code and in each downstream call.
ElastiCache	X-Ray subsegment, CloudWatch metrics	Lookup time, hit rate, evictions, memory pressure.
RDS	X-Ray subsegment, Performance Insights	Query time, top SQL, waits, connections.
SQS	X-Ray subsegment, queue metrics	Enqueue time; backlog age for the consumer.

1.6.2 Reading an X-Ray Trace

X-Ray assembles the segment and its subsegments into a single **trace**, identified by a trace ID such as 1-6740d8f2-3b9c0e1a7d2f4c5b6a8e9f01. The first part after the version number is the request start time in Unix epoch seconds, written in hexadecimal; the rest is random. Figure 1.13 shows the trace timeline for our request as the X-Ray console draws it.

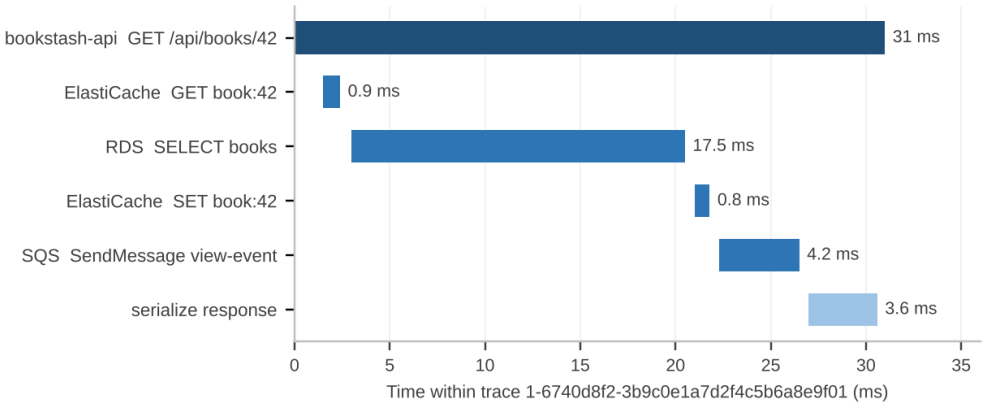


Figure 1.13 X-Ray timeline for one cold GET /api/books/42. The root segment covers the work inside the task; indented rows are subsegments.

Reading the timeline is straightforward once we know what to look for.

- **The root segment** (31 ms) is the whole time our task spent on the request. If this is small but the reader still waited a long time, the delay is outside our code: in DNS, the network, CloudFront or the ALB.
- **Gaps between subsegments** are time spent in our own code. The 0.6 ms gap before the RDS query is connection pool checkout and query preparation.
- **The longest subsegment** is usually where to start. Here the RDS query is 17.5 ms of 31 ms, which is expected for a cold path and confirms that the cache is worth having.
- **Parallel work** shows as overlapping bars. Our handler is sequential, so the bars do not overlap; if we fetched reviews and stock in parallel, they would.

X-Ray also builds a **service map** from many traces: a graph with a node for each service and edges for the calls between them, annotated with request rate, average latency and error percentage. The service map is the first screen we open during an incident, because a red node or edge points straight at the failing dependency.

> **KEY CONCEPT Trace, segment, subsegment, span**
>
> A **trace** is the complete record of one request across all instrumented components. A **segment** is the part recorded by one service (here, the BookStash API task). A **subsegment** is a unit of work inside a segment, usually a call to a dependency. In OpenTelemetry terms both segments and subsegments are **spans**, organised in a parent and child tree under one trace ID.

1.6.3 Instrumenting BookStash with OpenTelemetry

We instrument the API with the AWS Distro for OpenTelemetry. The Python service runs under the OpenTelemetry auto-instrumentation wrapper, which patches the web framework, the Redis client, the PostgreSQL driver and the AWS SDK so that their calls become spans without code changes. A collector runs as a second container in the same task and forwards spans to X-Ray. The relevant part of the task definition is shown below.

```
"containerDefinitions": [
  { "name": "api",
    "image": "111122223333.dkr.ecr.ap-south-1.amazonaws.com/bookstash-api:1.4.2",
    "command": ["opentelemetry-instrument", "gunicorn", "app:app", "-b", "0.0.0.0:8080"],
    "environment": [
      { "name": "OTEL_SERVICE_NAME", "value": "bookstash-api" },
      { "name": "OTEL_PROPAGATORS", "value": "xray,tracecontext" },
      { "name": "OTEL_PYTHON_ID_GENERATOR", "value": "xray" },
      { "name": "OTEL_EXPORTER_OTLP_ENDPOINT", "value": "http://localhost:4317" } ] },
  { "name": "otel-collector",
    "image": "public.ecr.aws/aws-observability/aws-otel-collector:latest",
    "command": ["--config=/etc/ecs/ecs-default-config.yaml"],
    "essential": false }
]
```

Two settings deserve comment. The X-Ray propagator makes the application understand the X-Amzn-Trace-Id header added by the ALB, so its spans join the same trace. The X-Ray ID generator produces trace IDs in the time-prefixed format that X-Ray expects. In a production system we would pin the collector image to a specific version rather than using latest, a point we return to in the CICD course.

1.6.4 Sampling

Tracing every request would cost money and add load for little extra insight. X-Ray therefore samples. The default sampling rule records the first request each second (the reservoir) and five percent of any additional requests. At BookStash's typical daytime rate of 40 requests per second, that is about 1 + 0.05 x 39, or roughly three traces per second, which is enough to see every recurring pattern. We add custom rules where we need more detail: for example, we trace 100 percent of POST /api/orders because orders are low in volume and high in importance.

1.6.5 Correlating Traces, Logs and Metrics

Traces are most useful when they connect to logs and metrics. We make that connection explicit in three ways. Every log line written by the API includes the trace ID. The ALB access logs carry the same ID. And CloudWatch dashboards for the API show a link from a latency spike to the traces recorded during that minute. When we have a trace ID from a reader's complaint, a CloudWatch Logs Insights query finds everything that happened for that request.

```
fields @timestamp, @logStream, level, message
| filter trace_id = "1-6740d8f2-3b9c0e1a7d2f4c5b6a8e9f01"
| sort @timestamp asc
```

EXAMPLE 1.8 Using a trace to find a slow query after a release

After release 1.5.0 the p95 TargetResponseTime alarm fired: the API took 410 ms at the 95th percentile against a budget of 200 ms. The service map showed the RDS edge in amber with an average latency of 260 ms; the ElastiCache edge was normal. Opening a slow trace showed one subsegment, SELECT ... FROM books WHERE lower(title) LIKE ..., taking 238 ms. That query was new in the release: the search feature had been changed to be case-insensitive. Performance Insights confirmed that it was now the top query by load and was scanning the whole books table. The fix was an expression index on lower(title), created in a few seconds. Fifteen minutes later p95 was back at 140 ms. The whole investigation used the evidence described in this section and took under twenty minutes.

EXAMPLE 1.9 Explaining a burst of 502 errors

On a Monday morning the ALB reported a short burst of HTTPCode_ELB_502_Count. There were no matching errors in the application logs and no failed X-Ray traces, which suggested that the requests never reached our code. The ALB access logs for those requests showed a target processing time of -1, meaning the connection to the target failed. The ECS service events showed a deployment at the same minute. The cause was a task that had been deregistered from the target group and stopped before in-flight connections drained, because the deregistration delay had been set to five seconds while some search requests took longer. Raising the delay to 30 seconds and making the application stop accepting new connections on the termination signal removed the errors. The lesson is that absence of evidence at one hop is itself evidence about where to look.

1.6.6 Tracing Asynchronous Work

Orders follow a path that a synchronous trace does not cover: the API puts a message on SQS and returns, and the worker processes it seconds later. SQS carries the trace header across this gap in a message system attribute named AWSTraceHeader. When the worker is instrumented in the same way as the API, its processing span is linked to the original request, and X-Ray shows the two parts of the order as connected work. The queue's ApproximateAgeOfOldestMessage metric then tells us how long orders wait between the two halves.

1.7 AWS Global Infrastructure

Everything in BookStash runs on physical machines in physical buildings. The way AWS organises those buildings into Regions, Availability Zones and edge locations determines where we can place our resources, how far apart we can spread them and what kinds of failure they can survive. At the time of writing (late 2026) AWS lists 39 launched Regions containing 123 Availability Zones, with more announced, and more than 750 CloudFront points of presence supported by 15 regional edge caches. These numbers grow every year; the structure behind them changes very little.

1.7.1 Regions

A **Region** is a separate geographic area where AWS operates a cluster of data centres. Each Region is designed to be isolated from every other Region: it has its own power and network connections, its own instances of most AWS services, and its own control systems. A Region is identified by a code such as ap-south-1 (Asia Pacific, Mumbai) or ap-south-2 (Asia Pacific, Hyderabad).

Isolation has direct consequences for the way we work. Resources we create live in the Region we create them in. An RDS database in ap-south-1 is not visible in ap-south-2. Data does not leave a Region unless we copy or replicate it, which also matters for regulation. Most service quotas and prices are set per Region. A small number of services are global, such as IAM, Route 53 and CloudFront, and their resources are not tied to a single Region.

Table 1.18 Criteria for choosing a Region for BookStash

Criterion	How it applies to BookStash
Latency to users	Most readers are in Telangana and Andhra Pradesh. Mumbai and Hyderabad are both close; CloudFront hides most of the difference for cached content.
Data residency and regulation	Readers' order data is personal data under India's Digital Personal Data Protection Act, 2023. Keeping it in an Indian Region simplifies compliance. Some sector rules, such as the Reserve Bank of India's 2018 direction on payment system data, require storage in India.
Service availability	New services and features often launch in a few Regions first. We confirm that every service we need is available in the chosen Region.
Price	Prices for the same resource differ between Regions. We compare using the AWS Pricing Calculator.
Recovery options	A second Region for backups should be far enough away to avoid shared disasters but within the same legal jurisdiction if regulation requires it.

For BookStash we chose ap-south-1 for the primary deployment, because it has the longest track record and the widest service catalogue among the Indian Regions, and ap-south-2 as the destination for cross-Region backup copies.

1.7.2 Availability Zones

An **Availability Zone (AZ)** is one or more discrete data centres within a Region, each with redundant power, networking and connectivity. The AZs in a Region are physically separated by a meaningful distance, so that a fire, flood or power failure in one is unlikely to affect another, but they are close enough (AWS states within about 100 km of each other) to be connected by redundant, high-bandwidth, low-latency private fibre. Round-trip latency between AZs is typically in the low single-digit milliseconds, which is fast enough for synchronous database replication.

Regions launched in recent years have at least three AZs; ap-south-1 and ap-south-2 each have three. The AZs are named by appending a letter to the Region code, as in ap-south-1a. Those letter names are mapped to physical zones independently for each AWS account, which spreads customers evenly. When two accounts need to refer to the same physical zone they use the **AZ ID**, such as aps1-az1, which is the same in every account.

```
$ aws ec2 describe-availability-zones --region ap-south-1 \
      --query "AvailabilityZones[].[ZoneName,ZoneId,State]" --output table
-----------------------------------------
|        DescribeAvailabilityZones        |
+--------------+-------------+------------+
|  ap-south-1a |  aps1-az1   |  available |
|  ap-south-1b |  aps1-az3   |  available |
|  ap-south-1c |  aps1-az2   |  available |
+--------------+-------------+------------+
```

The AZ is the main unit of fault isolation that we design around. Several AWS services take advantage of it automatically: an ALB has nodes in every zone we enable, SQS stores each message redundantly across zones, and an RDS Multi-AZ deployment keeps its standby in a different zone from its primary. For services that run our own code, such as ECS, we must tell the service which zones (through which subnets) to use. Figure 1.14 shows how BookStash is laid out across two zones.

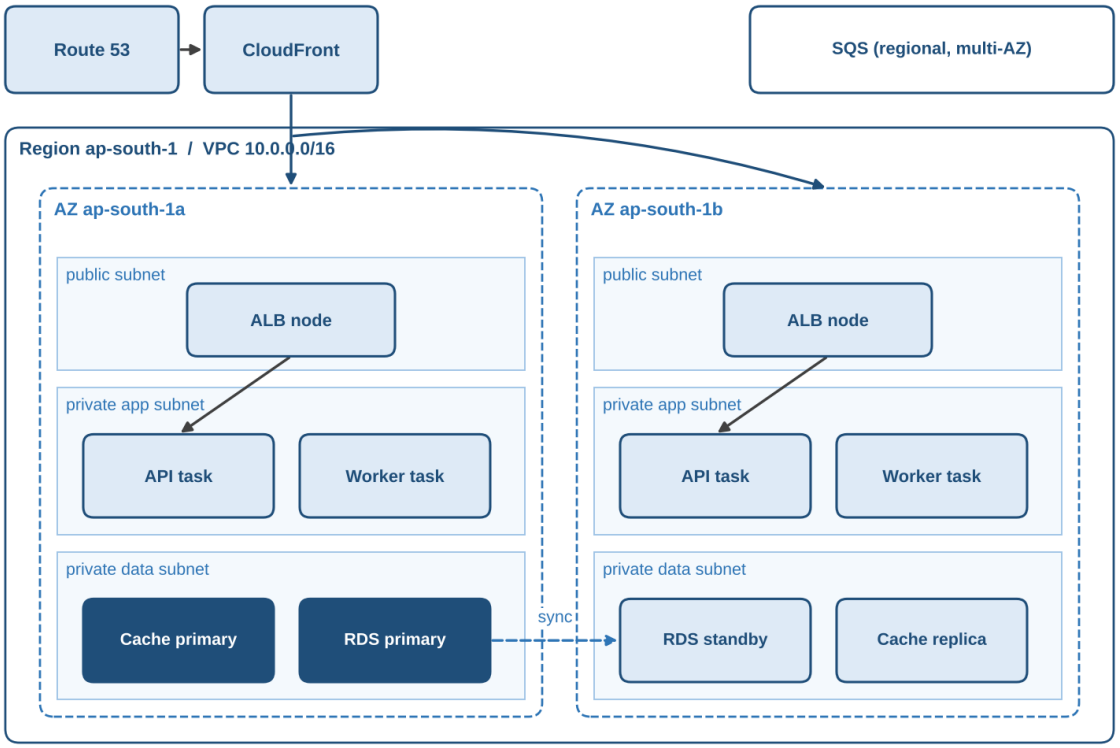


Figure 1.14 BookStash deployed across two Availability Zones in ap-south-1. Each zone has public, private application and private data subnets.

EXAMPLE 1.10 What happens to BookStash when one AZ fails

Assume ap-south-1a becomes unavailable at 14:00. The ALB stops routing to targets in that zone as soon as its health checks fail, within about 30 seconds with our settings. Requests continue to reach the API and worker tasks in ap-south-1b. ECS notices that the service is below its desired count and starts replacement tasks in the remaining healthy subnets. RDS detects the loss of the primary and promotes the synchronous standby in ap-south-1b; with Multi-AZ this usually completes in one to two minutes, during which database calls fail and the API returns errors for requests that need the database. ElastiCache promotes its replica in ap-south-1b. SQS is unaffected. The visible impact for readers is a short period of errors on uncached requests, while cached pages continue to be served by CloudFront. No order already placed on the queue is lost.

1.7.3 Edge Locations and Regional Edge Caches

Edge locations are points of presence in many more cities than there are Regions. They do not run general compute workloads. They host CloudFront caches, the Route 53 name servers that answer DNS queries, AWS Global Accelerator entry points and AWS WAF and Shield protection. In India there are edge locations in all major metropolitan areas, including Hyderabad, Chennai, Bengaluru, Mumbai, New Delhi and Kolkata.

Regional edge caches sit between the edge locations and the origin. They are larger and fewer, and they keep objects that are not popular enough to stay in every edge cache. A request that misses at the edge often hits at the regional cache, which shortens the path and reduces load on the origin, as we saw in Figure 1.5.

AWS also offers smaller extensions of a Region for special latency needs. **Local Zones** place a subset of services (compute, storage, some databases) in a metropolitan area close to users. **Wavelength Zones** place compute inside telecommunications providers' 5G networks. **AWS Outposts** puts AWS-managed hardware in a customer's own data centre. BookStash does not need any of these, but they appear in the design of media, gaming and factory systems.

1.7.4 The Geographic Hierarchy

Figure 1.15 draws the hierarchy for the two Indian Regions. Reading from the outside in: the AWS global network connects all Regions over private fibre; each Region contains several AZs; each AZ contains one or more data centres; edge locations sit outside the Regions, close to users.

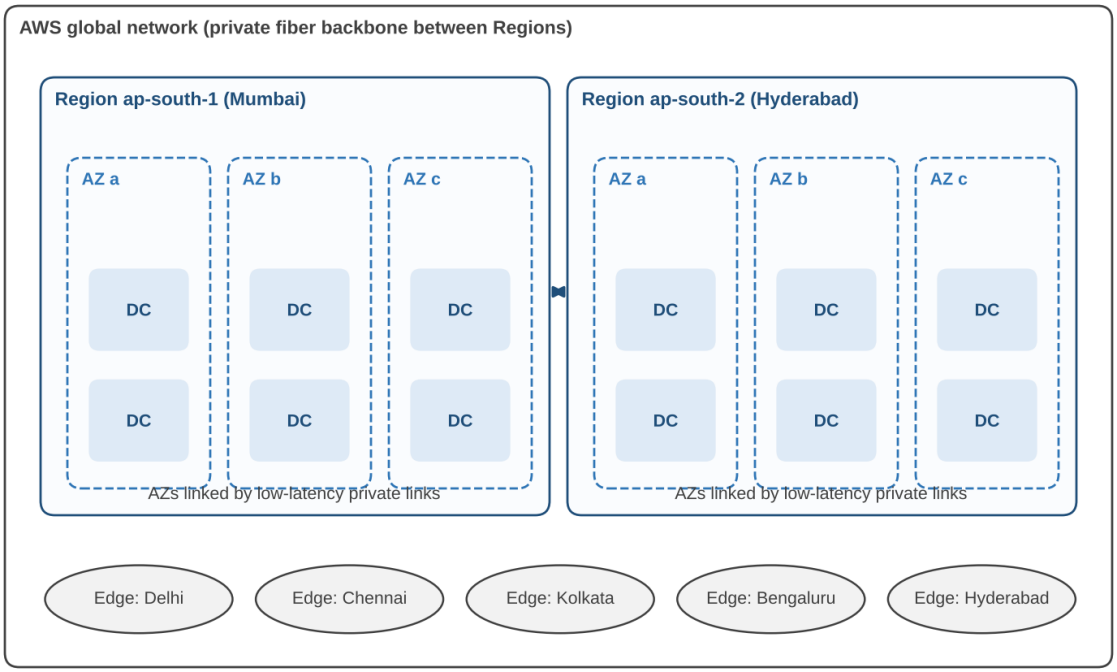


Figure 1.15 The geographic hierarchy for the two Indian AWS Regions and a selection of edge locations.

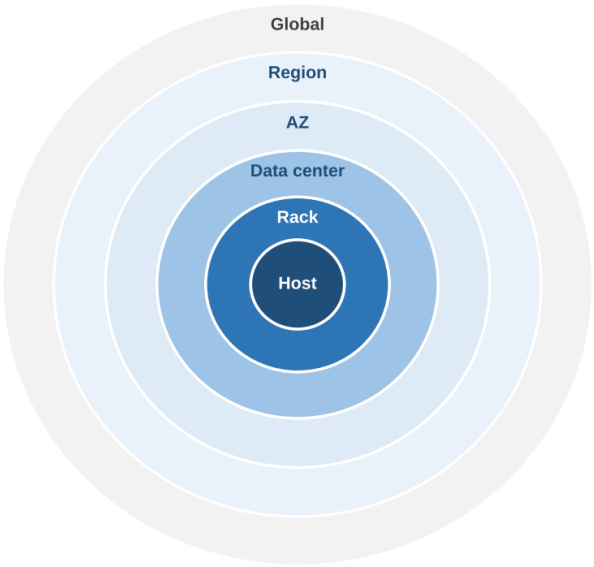
Table 1.19 AWS geographic building blocks

Building block	Scope	What we place there
Region	Geographic area with isolated infrastructure	VPCs, RDS, ElastiCache, SQS queues, ECS clusters, CloudWatch data
Availability Zone	One or more data centres in a Region	Subnets, Fargate tasks, RDS primary or standby, cache nodes
Data centre	Single building	Nothing directly; AWS manages placement
Edge location	Point of presence near users	CloudFront caches, Route 53 answers, WAF rules (configured globally)
Regional edge cache	Larger cache between edges and origins	Nothing directly; used by CloudFront automatically
Local Zone, Wavelength Zone, Outpost	Extensions of a Region	Latency-sensitive compute for special cases

1.7.5 The Failure-Domain Hierarchy

A **failure domain** is a set of components that can fail together because they share something: a power supply, a network switch, a building, a control system. The geographic hierarchy is also a

hierarchy of failure domains, from a single host up to the whole provider. Figure 1.16 shows the nesting and the mitigation we use at each level.



Domain	Typical failure	Our mitigation
Host	One task or instance lost	Run 2+ tasks behind the ALB
Rack / DC	Several hosts lost together	Spread tasks across AZs
AZ	Power, cooling or network event	Multi-AZ subnets, RDS Multi-AZ
Region	Rare, wide service event	Backups and DR in a second Region
Global	Shared control plane issue	Static stability, runbooks

Figure 1.16 Nested failure domains, from host to global, with the BookStash mitigation for each level.

Three principles follow from the hierarchy.

- **Spread across the domain you want to survive.** Two tasks on the same host survive a process crash but not a host failure. Two tasks in the same AZ survive a host failure but not a zone failure. BookStash runs tasks in two AZs because the AZ is the largest domain we design to survive automatically.
- **Make the cost of surviving each level explicit.** Surviving an AZ failure costs a second set of tasks and a database standby. Surviving a Region failure costs a second copy of the whole stack, or at least tested backups and a recovery plan in another Region, plus a DNS strategy to switch traffic. We decide which levels to cover based on the business impact of downtime.
- **Prefer data planes to control planes during recovery.** Many AWS services have a control plane (the APIs that create and change resources) and a data plane (the part that serves traffic). Data planes are usually simpler and more available. Some global services process changes through a single Region; for example, changes to IAM and to Route 53 records are handled by control planes in us-east-1, while their data planes are distributed worldwide. A recovery plan that depends on making such changes during an incident is weaker than one that relies on resources already in place, such as a Route 53 failover record with a health check. AWS calls this property **static stability**.

Table 1.20 Recovery objectives and the levels of protection they imply

Objective	Backup and restore	Pilot light / warm standby	Active-active multi-Region
Survives AZ failure	Yes, if Multi-AZ is used	Yes	Yes
Survives Region failure	Yes, after restore	Yes, after scale-up	Yes, immediately
Typical recovery time (RTO)	Hours	Tens of minutes	Seconds to minutes
Typical data loss (RPO)	Up to last backup	Seconds to minutes	Near zero to seconds
Relative cost	Low	Medium	High
BookStash today	Cross-Region snapshot copies	Not yet	Not needed

EXAMPLE 1.11 Setting RTO and RPO for BookStash

The bookstore manager tells us that losing the ability to take orders for half a day during term would be inconvenient but not serious, while losing confirmed orders would be serious because students pay at collection based on them. We translate this into a recovery time objective (RTO) of four hours for a Region failure and a recovery point objective (RPO) of 15 minutes for order data. RDS automated backups with point-in-time recovery, combined with automated copying of snapshots and transaction logs to ap-south-2, meet the RPO. A tested runbook that recreates the stack in ap-south-2 from templates and restores the database meets the RTO. The SDSR course examines multi-Region designs for systems whose objectives are much tighter.

1.8 Course Scope and Boundaries

A cloud system touches almost every area of software engineering. To keep this course focused we draw clear lines around what we cover and point to the companion courses for the rest. Figure 1.17 shows the three courses and how they connect.

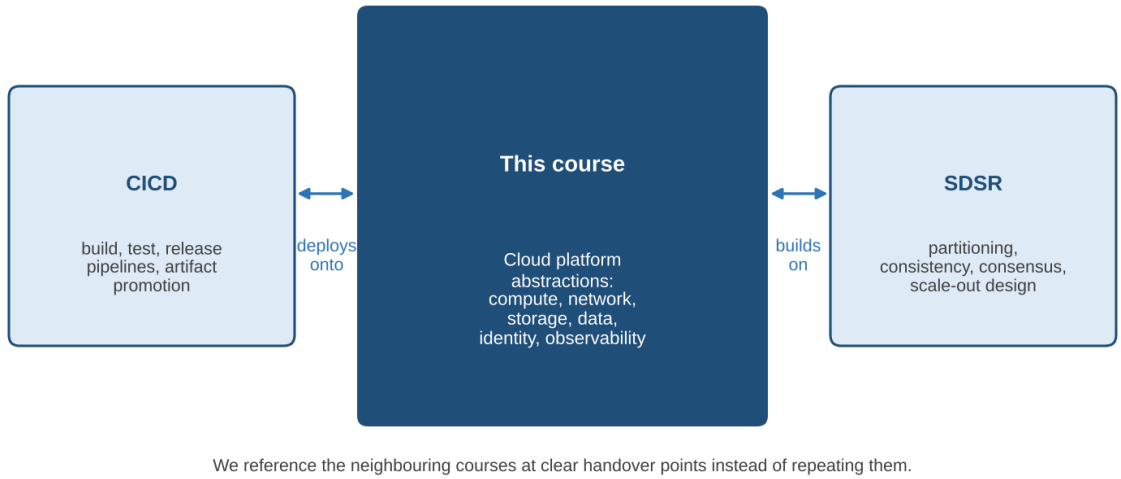


Figure 1.17 This course and its two neighbours. Each course owns a distinct layer of the problem and hands over at defined points.

1.8.1 Cloud-Platform Abstractions in This Course

This course is about the abstractions that a cloud platform provides and how to use them well. We study each tier of the canonical architecture in depth: networking (VPCs, subnets, routing, security groups, load balancers), compute (virtual machines, containers, functions), storage and databases (block, object, relational, in-memory, queues), identity and security (IAM, encryption, secrets), and operations (monitoring, logging, tracing, cost). For each abstraction we ask what it hides, what it guarantees, what it costs and how it fails. BookStash is the running example in every chapter.

1.8.2 CI/CD Pipelines in the CICD Course

How BookStash gets from source code to a running task is the subject of the CICD course. That course covers source control workflows, automated builds of container images, testing stages, artifact repositories, deployment strategies (rolling, blue and green, canary), infrastructure as code pipelines and release governance. In this course we use deployments that have already been built for us, and we refer to the CICD course whenever a topic depends on how changes are delivered.

1.8.3 Large-Scale Distributed-System Design in the SDSR Course

What BookStash would need to serve a hundred million readers, or to keep running through the loss of a whole Region with no visible impact, is the subject of the SDSR course. That course covers data partitioning and sharding, replication and consistency models, consensus, distributed transactions, back-pressure, multi-Region active-active designs and capacity planning at scale. In this course we stay within one Region and a scale that one well-designed managed stack can handle.

Table 1.21 Topic ownership across the three courses

Topic	This course	CICD	SDSR
VPC design, subnets, security groups	Primary	Uses	Uses
ECS, Fargate, Lambda configuration	Primary	Deploys to	Uses
RDS, ElastiCache, SQS usage and settings	Primary	Migrates schema	Extends
IAM roles and least privilege	Primary	Pipeline roles	Uses
Building and scanning container images	Mentions	Primary	Not covered
Blue and green and canary deployments	Mentions	Primary	Uses
Infrastructure as code tooling	Reads templates	Primary	Uses
Sharding, consistency models, consensus	Mentions	Not covered	Primary
Multi-Region active-active	Mentions	Not covered	Primary
Observability: metrics, logs, traces	Primary	Deployment signals	SLO design at scale

> **NOTE How to use the cross-references**
>
> When the text says that a topic is covered in CICD or SDSR, we give enough context to follow the current chapter without the other course. Students taking the courses in parallel will find the same BookStash system in all three, viewed from a different angle each time.

Chapter Summary

- We began with BookStash, a deployed multi-tier application on AWS, and followed a request from the reader's browser to the database and back before naming any component in detail.
- The canonical cloud-native architecture has a client, DNS (Route 53), a CDN (CloudFront), a load balancer (ALB), a stateless application tier (ECS on Fargate), a data tier (RDS, ElastiCache, SQS) and observability across all tiers (CloudWatch, X-Ray).
- Service models (IaaS, CaaS, PaaS, FaaS, SaaS) differ in how many layers the provider manages. Moving up trades control for reduced operational work but never removes customer responsibility.
- Under the shared responsibility model AWS secures and makes resilient the cloud itself; customers secure and make resilient what they build in it. The boundary moves with the service.
- Azure and GCP offer close equivalents for every BookStash tier, with differences in load balancing, serverless containers, network scope and queue semantics.
- End-to-end tracing follows one request hop by hop. CloudFront, Route 53 and the ALB leave evidence in logs and metrics; our task produces X-Ray segments and subsegments linked by the trace header.
- AWS infrastructure is organised into Regions, Availability Zones and edge locations. This hierarchy is also a hierarchy of failure domains, and we design to survive the loss of an AZ automatically.
- This course covers cloud-platform abstractions; CI/CD pipelines and large-scale distributed design belong to the CICD and SDSR courses.

Key Terms

Table 1.22 Key terms introduced in this chapter

Term	Definition
Alias record	A Route 53 record that points a name, including a zone apex, directly at an AWS resource such as a CloudFront distribution.
Availability Zone (AZ)	One or more discrete data centres in a Region with independent power and networking, connected to other AZs by low-latency links.
Cache-aside	A pattern in which the application reads from a cache, falls back to the database on a miss and writes the result into the cache.
Cache hit ratio	The fraction of requests answered from a cache without contacting the origin.
Cloud-native	Built from replaceable stateless processes, managed data services, declarative deployment and built-in observability.
Control plane	The part of a service that creates and changes resources, as opposed to the data plane that serves traffic.

Term	Definition
Edge location	A point of presence close to users that hosts CloudFront caches and Route 53 name servers.
Failure domain	A set of components that can fail together because they share a dependency.
Health check	A periodic probe that decides whether a target should receive traffic.
Multi-AZ	A deployment that keeps resources or replicas in more than one AZ so that the loss of one zone is survivable.
Region	An isolated geographic area containing multiple AZs.
Segment and subsegment	X-Ray records of the work done by one service and by its calls to dependencies within a trace.
Service model	A classification (IaaS, CaaS, PaaS, FaaS, SaaS) of which layers a provider manages.
Shared responsibility model	The division of security and resilience duties between the cloud provider (of the cloud) and the customer (in the cloud).
Span	The OpenTelemetry term for a timed unit of work within a trace.
Static stability	The property of continuing to operate through a failure without needing to make control-plane changes.
Target group	A set of ALB targets with a shared health check and routing configuration.
Task (ECS)	One or more containers scheduled together by ECS; the unit that Fargate runs.
Trace	The complete record of one request across all instrumented components, identified by a trace ID.
TTL (time to live)	How long a DNS answer or cached object may be reused before it must be fetched again.
Visibility timeout	The period during which an SQS message received by one consumer is hidden from others.

Self-Test Questions with Answers

The questions in this section are answered strictly from the material in this chapter. Try each one before reading the answer.

1. Why does CloudFront reduce latency for a reader even on a cache miss?

> **Answer.** Because the TCP and TLS handshakes complete at a nearby edge location, and the edge reuses long-lived connections over the AWS backbone to reach the origin. The reader avoids a full handshake across the long path.

2. Name the three data services in BookStash and the access pattern each serves.

Answer. RDS for PostgreSQL is the durable system of record for books, stock and orders. ElastiCache holds hot book records in memory for fast repeated reads. SQS buffers orders and events so that the API can reply at once and workers can process later.

3. In the shared responsibility model, who is responsible for enabling encryption on an RDS instance, and why?

Answer. The customer. AWS provides the encryption capability, but whether it is enabled is a setting in the customer's account, and settings the customer controls are the customer's responsibility.

4. Which BookStash hops produce X-Ray spans, and where is evidence found for the others?

Answer. The Fargate task produces the segment and subsegments for ElastiCache, RDS and SQS calls. Route 53 evidence is in query logs, CloudFront evidence in access logs and metrics, and ALB evidence in access logs that carry the trace ID.

5. What is the difference between an AZ name and an AZ ID?

Answer. An AZ name such as ap-south-1a is mapped to a physical zone separately for each account. An AZ ID such as aps1-az1 identifies the same physical zone in every account.

6. Why is the BookStash cache not counted in the availability chain for correctness?

Answer. Because if ElastiCache fails the API falls back to reading from RDS. The request is slower but still succeeds, so a cache failure does not make the request fail.

7. Give one reason to choose FaaS and one reason to avoid it, using BookStash features.

Answer. Cover thumbnail generation suits FaaS because it is short, event-driven and idle most of the time. The book API does not, because it runs continuously, holds connection pools and would face cold starts.

8. What does static stability mean, and how does BookStash apply it to DNS?

Answer. It means a system keeps working through a failure without needing control-plane changes. A Route 53 failover record with a health check, created in advance, switches traffic without anyone editing records during the incident.

Review Questions

1. Draw the BookStash architecture from memory and label each component with its role in one phrase.
2. Explain why a request's total latency includes time spent outside the application's code, using the waterfall in Figure 1.2.
3. List the five NIST essential characteristics and give one BookStash example of each.
4. Compare IaaS and CaaS for the BookStash API in terms of what the team must manage.
5. Explain the phrase security of the cloud versus security in the cloud with two examples of each.
6. Why is an SLA not the same as a guarantee that a service will not fail?
7. Describe two ways in which Azure Front Door differs from the CloudFront and ALB combination.
8. What does the X-Amzn-Trace-Id header do, and which component adds it in BookStash?
9. Explain how an SQS message's visibility timeout protects an order when a worker crashes.
10. Why do Regions launched in recent years have at least three AZs rather than two?
11. Explain the difference between a control plane and a data plane with an AWS example.
12. For each of the CICD and SDSR courses, name two topics that this course deliberately leaves to them.

Exercises

1. **Latency budget.** A reader in Guwahati reports 600 ms response times for /api/books/42. Using Table 1.3 as a template, list the measurements you would collect for each hop and the tools that provide them.
2. **Cache sizing.** BookStash has 80,000 books. Each cached record is about 1.2 KB. Estimate the memory needed to cache every book, then explain why caching only the most popular 10 percent may give nearly the same hit ratio.
3. **Availability arithmetic.** Recompute the chain availability in Example 1.6 for a design where RDS runs as a single instance with an assumed availability of 99.5 percent. How many minutes of downtime per month does this add?
4. **Responsibility mapping.** Extend Figure 1.10 with a column for Amazon DynamoDB. Justify each cell.
5. **Multi-cloud port.** Using Table 1.13 and Table 1.14, write a one-page porting plan for BookStash on Azure, listing every change to code and configuration.
6. **Failure drill.** Write a step-by-step runbook for the AZ failure described in Example 1.10, including the metrics you would watch and the point at which you would declare recovery.

Hands-On Lab 1: Observing BookStash End to End

Objective. Observe every tier of the deployed BookStash application and connect one request across DNS, CDN, load balancer, task, cache and database.

Prerequisites. Read-only access to the course AWS account, the AWS CLI configured for ap-south-1, and a terminal with dig and curl.

1. Resolve bookstash.com with dig and record the addresses. Run whois or a reverse lookup on one address and confirm it belongs to CloudFront.
2. Request /covers/42.jpg twice with curl -I. Record x-cache, x-amz-cf-pop and age on both requests and explain any difference.
3. Request /api/books/42 three times, one second apart. Note which responses were hits and which reached the origin.
4. Copy the trace ID from an origin response. In the X-Ray console, open the trace and record the duration of each subsegment.
5. Run the CloudWatch Logs Insights query from Section 1.6.5 with your trace ID and identify the log lines written for your request.
6. Open the ALB access logs for the same minute and find your request by trace ID. Record request, target and response processing times.
7. In the ECS console, list the running API tasks and their Availability Zones. Confirm that tasks run in at least two zones.
8. Open the RDS console and confirm whether the instance is Multi-AZ, which zone the primary is in, and the backup retention period.
1. Write a half-page report that reconstructs your request from browser to database and back, with a time for every hop you could measure.

Expected outcome. A timed, hop-by-hop account of one real request, and a list of the evidence sources used for each hop.

Further Reading

- Mell, P. and Grance, T. (2011). The NIST Definition of Cloud Computing. NIST Special Publication 800-145. National Institute of Standards and Technology.
- Amazon Web Services. AWS Well-Architected Framework, including the Reliability Pillar and the Security Pillar. Available from the AWS documentation site.
- Amazon Web Services. Shared Responsibility Model. Available from the AWS compliance pages.
- Amazon Web Services. AWS X-Ray Developer Guide, section on migrating from X-Ray instrumentation to OpenTelemetry.

- Amazon Web Services. AWS Global Infrastructure: Regions and Availability Zones.
- Beyer, B., Jones, C., Petoff, J. and Murphy, N. R. (eds.) (2016). Site Reliability Engineering: How Google Runs Production Systems. O'Reilly Media.
- Kleppmann, M. (2017). Designing Data-Intensive Applications. O'Reilly Media.
- Cloud Native Computing Foundation. CNCF Cloud Native Definition v1.1.